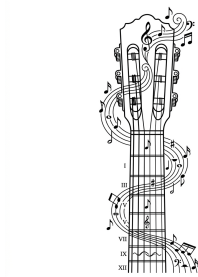


Interpretable Difficulty Estimation for Classical Guitar from Symbolic Notation

Aser Abdelhakeem

A Bachelor's Thesis submitted in partial fulfillment of the
requirements for the degree of Bachelor of Science in Computer
Science



Department of Computer Science

University Name

May 2026

Abstract

Matching repertoire to a student’s technical level is a routine task in classical guitar teaching, yet the difficulty grades in repertoire databases are assigned manually and come with no explanation of why a piece earned its grade. Automatic difficulty estimation from symbolic scores has shown that interpretable prediction is possible, but existing work targets the piano, and its descriptors do not transfer to the guitar, where one staff encodes both hands and the same pitch can be played at several fretboard positions.

This thesis adapts RubricNet, an interpretable additive neural model, from piano to classical guitar. The model scores 32 hand-crafted descriptors independently and sums them, so every prediction decomposes into per-descriptor contributions like a grading rubric. The descriptors span left-hand demands (stretches, shifts, barre chords), right-hand demands (string crossing, plucking patterns), and global score properties (length, polyphony, note density). We assemble a dataset of 716 pieces, pairing symbolic scores from three sources with 1–20 grades from the GuitarBurst database; because PDF-derived scores lack rhythm, a Needleman–Wunsch alignment transplants note timing from matched MIDI performances, giving 89% of the dataset usable rhythm information and enabling speed- and passage-aware descriptors.

On an 8-class ordinal task, the model reaches 31.0% exact accuracy. A Random Forest baseline edges it on raw accuracy (33.1%) and performs about the same on balanced accuracy (~ 0.29), but the interpretable model wins on the metrics that matter for ordinal prediction — lower mean absolute error (1.069 vs. 1.122) and higher rank correlation (Kendall’s τ 0.634 vs. 0.599) — so when it errs, it errs by less. Two fuzzy rule-based classifiers, adapted from music genre classification as a second interpretable reference point, trail far behind on the same descriptors (Kendall’s τ at

most 0.48): the model’s standing comes not from interpretability being cheap, but from combining it with an ordinal design. Accuracy rises to 67.6% on a three-level easy/medium/hard scale. Per-descriptor contributions increase monotonically with difficulty and agree with Random Forest importances, supporting the trained model as a difficulty rubric grounded in guitar technique rather than an opaque predictor.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	3
1.3	Research Questions	4
1.4	Research Goals	5
1.5	Thesis Structure	6
2	Related Work	7
2.1	Symbolic Difficulty Estimation	7
2.2	Multi-Dimensional Difficulty Representation	8
2.3	The RubricNet Model	8
2.4	Fuzzy Rule-Based Music Classification	9
2.5	Guitar Playability Metrics	10
2.6	Datasets	10
3	Preliminaries	12
3.1	The Classical Guitar	12
3.2	Symbolic Music Representations	14
3.3	Global Sequence Alignment	15
3.4	Ordinal Classification	15
3.5	Baseline Models	16
4	Method	18
4.1	Dataset Construction	18
4.1.1	Difficulty Labels	18

4.1.2	Symbolic Score Sources	19
4.1.3	Matching and Deduplication	19
4.1.4	Difficulty Binning	19
4.1.5	Cross-Validation Splits	21
4.2	Rhythm Alignment	21
4.3	Descriptor Engineering	23
4.3.1	V1: Initial Descriptor Set	23
4.3.2	V2: Audited and Expanded Set	24
4.3.3	V3: Rhythm-Aware Descriptors	25
4.4	Model Architecture and Training	26
4.4.1	Additive Subnetworks	27
4.4.2	Ordinal Prediction Head	27
4.4.3	Ordinal Label Smoothing	28
4.4.4	Training Procedure and Hyperparameter Search	28
4.5	Fuzzy Rule-Based Comparison Methods	29
4.5.1	Fuzzification	29
4.5.2	Complete Search of Primitive Rules	30
4.5.3	Fuzzy Pattern Trees	31
5	Evaluation	32
5.1	Evaluation Setup	32
5.2	Quantitative Analysis	33
5.2.1	8-Class Difficulty Estimation	33
5.2.2	Per-Dimension Ablation	36
5.2.3	Coarse 3-Class Evaluation	37
5.3	Qualitative Analysis	37
5.3.1	Descriptor Influence	37
5.3.2	Monotonicity	39
5.3.3	Context-Free vs. Context-Aware Descriptors	41
5.3.4	Fuzzy Rules as Difficulty Descriptions	43
5.4	Auxiliary Experiments	45

5.4.1	Direct 20-Level Classification	45
5.4.2	Pruning Weak Descriptors	45
5.4.3	Ordinal Label Smoothing	46
5.4.4	Multi-Task Coarse Auxiliary Head	46
5.4.5	Refined Left-Hand Descriptors (V4)	46
5.4.6	Rhythm Ablation and Interaction Descriptors	47
5.4.7	Fuzzy Rule Ablations	49
6	Discussion	51
6.1	Descriptor Design	51
6.2	Predictive Performance	52
6.3	The Multi-Dimensional Nature of Guitar Difficulty	54
6.4	Interpretability	54
7	Conclusion	56
7.1	Future Work	58

List of Figures

3.1	The fretboard coordinate system: strings s (with their open-string pitches in standard tuning) and frets f , with fret 0 denoting the open string. The four marked positions — $(1, 0)$ open, and $(2, 5)$, $(3, 9)$, $(4, 14)$ fretted — all produce the same pitch E4; choosing among them is a fingering decision with direct consequences for playability.	13
4.1	Distribution of the raw GuitarBurst difficulty levels over the 716-piece dataset. Dashed lines mark the boundaries of the 8 ordinal classes produced by equal-frequency binning; alternating shades indicate bin membership. The sparsity of the highest levels (levels 17–20 hold 7, 8, 5, and 3 pieces) is what makes direct 20-level classification untenable (Section 5.4.1).	20
4.2	Rhythm alignment, schematically. The PDF-derived score contributes the pitch sequence but no timing (top); the matched MIDI performance contributes onset times (bottom). Needleman–Wunsch alignment pairs the two note sequences, absorbing local insertions such as the ornamental F#4 as gaps, and each PDF note receives the inter-onset interval of its MIDI counterpart as its duration.	22
4.3	The RubricNet architecture. Each descriptor x_i is scored independently by a two-parameter subnetwork g_i ; the bounded scores are summed into the aggregated difficulty score $S(x)$, and an ordinal head maps $S(x)$ to K cumulative class probabilities. No descriptor interacts with any other before the summation, so $g_i(x_i)$ is exactly descriptor i ’s contribution to the prediction.	27

4.4	The five triangular membership functions over the normalized descriptor range. Each normalized value belongs to at most two adjacent linguistic terms, with memberships summing to 1.	30
5.1	Confusion matrix of RubricNet V3 test predictions (seed 0, pooled over the five test folds; cell counts annotated, rows normalized by true-class size). Errors concentrate within one bin of the diagonal. . .	35
5.2	Descriptor importance under three independent measures: RubricNet influence ranges, Random Forest feature importances, and absolute Spearman correlations with the raw difficulty labels (V3 descriptors, fold 0).	39
5.3	Mean per-descriptor subnetwork output by true difficulty class (V3 model, fold 0). Contributions rise steadily with difficulty for the influential descriptors; <code>open_string_ratio</code> falls, consistent with open strings making music easier to play.	40
5.4	Per-descriptor contributions $g_i(x_i)$ for an easy and a hard test piece (V3 model, fold 0), sorted by the Chaconne's contributions. The summed scores $S(x) = -3.06$ and $+5.66$ place the pieces in classes 0 and 6, respectively; every bar is one line of the rubric behind those predictions.	42

List of Tables

4.1	Dataset source distribution.	20
5.1	8-class difficulty estimation across descriptor generations (mean $\pm$ std; RubricNet V2/V3 over 3 seeds $\times$ 5 folds, RubricNet V1 over a single seed, baselines over 5 folds). Bold marks the best value per generation.	34
5.2	Per-dimension ablation with Random Forest on V3 descriptors (mean $\pm$ std over 5 folds).	36
5.3	Coarse 3-class evaluation (Easy / Medium / Hard), obtained by mapping 8-class predictions.	37
5.4	Context-free descriptors vs. their rhythm-aware counterparts: Spearman ρ against raw difficulty on the 639 pieces with rhythm data. . . .	43
5.5	Top-5 complete-search rules by relevance (Equation 4.6) for the easiest and hardest difficulty classes, fold 0.	44
5.6	Effect of interaction descriptors: RubricNet, ordinal regression, and Random Forest all declined when interaction features were appended to the V3 base descriptors.	48

Chapter 1

Introduction

1.1 Motivation

The classical guitar is taught, almost universally, through graded repertoire: a student progresses through pieces of increasing technical demand, and the teacher’s central curricular task is to select works that lie within the student’s current reach while still advancing their technique. This selection shapes lesson planning, the order in which techniques are introduced, how efficiently practice time is spent, and, not least, whether the student remains motivated or becomes discouraged by a piece encountered too early. The difficulty judgments underlying these decisions are made constantly in guitar pedagogy, yet they rest almost entirely on individual teacher experience and informal consensus. Two experienced teachers may disagree about the same piece, and neither can appeal to a shared, reproducible standard to settle the question.

Graded repertoire databases partially address this need. GuitarBurst [8], for example, assigns classical guitar works an integer difficulty grade from 1 to 20, and the existence and popularity of such resources demonstrates a genuine demand for structured difficulty information. However, these grades are assigned manually, piece by piece, and the databases offer no account of *why* a piece received its grade — which technical properties of the score pushed it to level 12 rather than level 8. Manual grading also does not scale: the volume of available repertoire, particularly in public-domain score archives, far exceeds what any curated database covers.

At the same time, research in music information retrieval (MIR) has shown that symbolic score representations, such as MusicXML and MIDI, carry sufficient signal for automatic difficulty estimation. Yaroch [26] estimated piano score difficulty from symbolic features extracted from MusicXML, and Ramoneda et al. [15] demonstrated that piano difficulty can be predicted by an interpretable model built from a small set of hand-crafted descriptors, organized so that the model’s output reads like a pedagogical rubric: each descriptor receives an independent score, and the scores are summed into the final difficulty estimate. Interpretability is not an incidental property in this setting. A difficulty estimate is only actionable in a pedagogical context if the teacher can inspect where it comes from and verify the reasoning against the score.

Nearly all of this work, however, targets the piano. The descriptors of RubricNet, the strongest interpretable model to date, are keyboard-specific: they assume separate staves per hand, key-span measurements, and piano fingering conventions, none of which transfer to the guitar, where a single staff encodes the work of both hands and the same pitch can be realized at several positions on the fretboard. On the guitar side, Vélez Vásquez et al. [25] formalized the playability of strummed chord progressions in pop music through descriptors such as barre usage and chord-transition difficulty, but solo classical repertoire — with its contrapuntal textures, position shifts along the neck, and intricate right-hand arpeggiation — has no comparable interpretable difficulty model.

This thesis addresses that gap. We adapt the RubricNet framework to classical solo guitar: the descriptor layer is redesigned around the instrument’s actual technical demands, a dataset pairing symbolic scores with professional difficulty grades is assembled from scratch, and the resulting model is evaluated both on predictive performance and on whether its explanations align with how guitarists reason about difficulty. The aim is not merely a classifier that assigns a level, but one that can justify the level in terms a teacher would recognize — stretch, shifting, barre load, texture — and do so from the score alone.

1.2 Problem Statement

Transferring interpretable difficulty estimation from the piano to the classical guitar poses several challenges that go beyond substituting one instrument’s features for another’s.

The first challenge concerns data. No existing resource pairs high-quality symbolic classical guitar scores with professional difficulty grades at a scale suitable for supervised learning. Graded databases such as GuitarBurst provide labels but no machine-readable scores; symbolic corpora such as DadaGP [18] and GAPS [17] provide scores but no difficulty grades. A dataset must therefore be constructed by matching heterogeneous score sources against an external grading resource, which raises problems of title and composer ambiguity, cross-source duplication, and inconsistent file quality. The difficulty is compounded by the largest available score source, PDF tablature: parsing these vector-graphic documents recovers pitch, string, and fret reliably, but note durations are lost entirely. Without rhythm, any descriptor involving timing — how fast a passage must be executed, how much time a position shift has available — cannot be computed, even though execution speed is a central determinant of physical difficulty.

The second challenge concerns descriptor design under an interpretability constraint. The additive architecture that makes RubricNet transparent is also its principal restriction: no descriptor may interact with any other before the final summation, so the model cannot learn on its own that, for instance, a wide left-hand stretch is more demanding when it must be executed quickly. If such an interaction matters — and in guitar technique it plainly does, since a five-fret jump during a fermata is trivial while the same jump inside a sixteenth-note run is not — it must be encoded into a descriptor by hand. Conversely, uninformative descriptors are not merely useless to an additive model: because every input is assigned a score and summed into the output, a dead descriptor injects noise directly into the prediction. Descriptor engineering, guided by systematic auditing, is therefore where the principal modeling effort of this work lies.

The third challenge concerns the labels themselves. Difficulty grades are ordinal

— level 12 is harder than level 8, and misclassifying a piece by one level is a far smaller error than misclassifying it by four — so both the model and the evaluation must respect the ordering rather than treat levels as unrelated categories. Moreover, the raw 1–20 scale is heavily imbalanced: intermediate levels are well populated while the most advanced levels contain only a handful of pieces each, which makes direct 20-way classification statistically untenable at the achievable dataset size and forces a principled coarsening of the label space. Finally, all grades originate from a single database, effectively a single grading tradition, so any evaluation is necessarily relative to that standard.

1.3 Research Questions

The central objective of this thesis is to develop and validate an interpretable difficulty estimation model for classical guitar. The following research questions guide the investigation:

1. **Which guitar-specific symbolic descriptors best capture the technical difficulty of classical guitar pieces, as reflected in graded repertoire resources such as GuitarBurst?**

The first question addresses the descriptor layer: which measurable properties of a symbolic guitar score — left-hand stretches, position shifts, barre chords, right-hand string crossings, texture, length — actually carry signal about the grades assigned by an established grading resource, and which intuitively plausible properties do not.

2. **Can an interpretable, rubric-based model using these hand-crafted descriptors predict piece-level classical guitar difficulty with useful accuracy?**

The second question concerns the cost of interpretability: whether a strictly additive model, restricted to scoring each descriptor independently, can remain competitive with unconstrained black-box baselines that are free to exploit arbitrary feature interactions. Its counterpart is the value of the model’s

ordinal design, probed by comparing against fuzzy rule-based classifiers that are equally interpretable but treat the difficulty levels as unrelated categories.

3. **Does combining multiple dimensions of difficulty — left-hand technique, right-hand technique, and global score complexity — improve prediction over any single dimension alone?**

The third question tests whether guitar difficulty is genuinely multi-dimensional. If a single aspect of technique sufficed to grade a piece, the multi-dimensional descriptor design would be unnecessary complexity.

4. **To what extent do the model’s descriptor contributions align with common pedagogical intuitions about what makes a classical guitar piece difficult?**

The final question examines whether the model’s interpretability is substantive rather than merely architectural: whether the learned per-descriptor contributions behave consistently across difficulty levels and agree with how teachers reason about technical demand.

1.4 Research Goals

To address the challenges identified in the research questions, this thesis pursues the following goals:

1. **Construct a labeled symbolic dataset for classical guitar difficulty.**

Assemble a corpus pairing symbolic scores from heterogeneous sources with professional 1–20 difficulty grades, including a rhythm-recovery pipeline that restores note durations to PDF-derived scores by aligning them against matched MIDI performances.

2. **Define a compact set of guitar-specific, interpretable descriptors.**

Develop descriptors computable directly from symbolic notation, organized into left-hand, right-hand, and global dimensions, and audit them systematically so that only informative inputs reach the additive model.

3. **Build and benchmark an interpretable difficulty classifier.**

Adapt the additive RubricNet architecture to the guitar descriptor set and compare it against both black-box baselines and interpretable fuzzy rule-based classifiers under a fixed ordinal evaluation protocol, testing in particular whether the multi-dimensional design outperforms each dimension in isolation.

4. **Validate the model’s explanations.**

Demonstrate that the trained model’s per-descriptor contributions are pedagogically meaningful, by analyzing their influence, their monotonicity across difficulty levels, and their agreement with independent measures of feature importance.

A substantial part of the work, implicit in the first goal but unavoidable in practice, is data engineering: since no suitable dataset existed, the dataset itself — including the rhythm-alignment pipeline — constitutes a contribution of this thesis.

1.5 Thesis Structure

In Chapter 2, we review prior work on symbolic difficulty estimation, multi-dimensional difficulty modeling, the RubricNet architecture, fuzzy rule-based music classification, and guitar-specific playability metrics. In Chapter 3, we introduce the necessary preliminaries: the technical vocabulary of classical guitar playing, symbolic music representations, global sequence alignment, ordinal classification, and the baseline models used for comparison. In Chapter 4, we describe our method, including the construction of the dataset, the rhythm-alignment pipeline, the three generations of descriptor sets, the model architecture together with its training procedure, and the two fuzzy rule-based comparison methods adapted to the task. In Chapter 5, we evaluate the approach quantitatively against baseline models and qualitatively through an analysis of the trained model’s internals, and report a series of auxiliary experiments. In Chapter 6, we discuss the findings with respect to the research questions and the limitations of the approach. Finally, in Chapter 7, we summarize our contributions and outline directions for future work.

Chapter 2

Related Work

This chapter reviews prior research relevant to the challenges this thesis addresses. First, we examine work on estimating musical difficulty from symbolic score representations, which establishes that symbolic notation carries usable difficulty signal but also exposes recurring weaknesses in label provenance and model transparency. Second, we review the argument that difficulty is a multi-dimensional quantity, which directly motivates the organization of our descriptor set. Third, we describe the RubricNet architecture, whose additive structure forms the methodological core of this thesis. Fourth, we review fuzzy rule-based approaches to interpretable music classification, from which two additional baselines used in the evaluation are drawn. Finally, we survey guitar-specific playability metrics and the datasets from which our corpus is assembled.

2.1 Symbolic Difficulty Estimation

Automatic difficulty assessment from musical scores is a small but growing subfield of music information retrieval. Yaroch [26] estimated piano score difficulty on a measure-by-measure basis from MusicXML, extracting symbolic features such as note density, interval sizes, and basic rhythmic properties, and training linear regression models and convolutional neural networks on them. The study demonstrated that symbolic notation alone carries usable difficulty signal, but it also illustrates two recurring weaknesses of the area: the difficulty labels were self-assigned rather than

drawn from an established grading system, and the better-performing models were black boxes that could not explain their predictions. Both issues are consequential in a pedagogical setting, where a difficulty estimate is only actionable if a teacher can trust its provenance and verify its reasoning. This thesis addresses the first weakness by grading against an established external repertoire database, and the second by adopting an interpretable-by-construction architecture.

2.2 Multi-Dimensional Difficulty Representation

Difficulty is not a single quantity. A piece can be hard to read but easy to execute, or technically demanding in one hand while trivial in the other. Ramoneda et al. [16] make this argument explicit for piano repertoire, decomposing difficulty into notation (reading complexity), technique (physical execution), and expressiveness (interpretative demands), and showing that estimators trained on the separate dimensions and then combined outperform a flat, single-vector feature representation. This finding directly shapes the descriptor design in this thesis: rather than pooling all features, the guitar descriptors are organized into left-hand technique, right-hand technique, and global score complexity, and Chapter 5 tests whether the combination in fact outperforms each dimension in isolation.

2.3 The RubricNet Model

The methodological core of this thesis is RubricNet, introduced by Ramoneda et al. [15] as a white-box ordinal classifier for piano difficulty on the CIPI dataset. Its defining property is an additive structure: each hand-crafted descriptor passes through its own small subnetwork, and the scalar outputs are summed into a single difficulty score,

$$S(x) = \sum_{i=1}^M g_i(x_i), \quad (2.1)$$

where x_i is the i -th descriptor and g_i its subnetwork. Because no descriptor interacts with any other before the summation, the contribution of each one to the final score can be read off directly from the trained model. The design deliberately mirrors a

pedagogical rubric, in which a teacher scores independent aspects of a performance and adds the scores together.

RubricNet achieved accuracy competitive with black-box baselines on piano data while remaining fully transparent. Its descriptors, however, are keyboard-specific — separate left- and right-hand staves, key spans, piano fingering patterns — and cannot be applied to guitar scores, where a single staff encodes the work of both hands and the same pitch can be played at several fretboard positions. Adapting the architecture therefore requires redesigning the entire descriptor layer, which constitutes the main technical work of this thesis. To the best of our knowledge, this is the first application of the additive rubric architecture outside piano repertoire.

2.4 Fuzzy Rule-Based Music Classification

The additive rubric is not the only route to interpretable music classification. A separate line of work builds classifiers from *fuzzy rules* [27] — natural-language statements of the form “if fret entropy is very high, then the piece is hard” — whose interpretability lies in the rules themselves rather than in a decomposition of a numeric score. Vatolkin and Rudolph [24] introduced this approach to music categorization, inducing primitive rules over high-level audio features by an exhaustive relevance-ranked search and showing that the resulting rule bases remain readable by non-experts. Heerde et al. [9] subsequently compared three fuzzy rule-based approaches on music genre classification: the complete search over primitive rules, an evolutionary optimization of entire rule bases, and fuzzy pattern trees [10, 19, 20], which compose fuzzy statements with logical operators into per-class trees. All three achieved usable genre accuracy while producing rules a listener can inspect directly.

This line of work matters to this thesis for two reasons. First, it offers a second, independently developed paradigm of interpretability against which RubricNet can be compared: readable rules over named features, rather than summed per-descriptor scores. Second, the methods are class-agnostic — they treat labels as unordered categories — which makes them a natural instrument for isolating the value of RubricNet’s ordinal design. Section 4.5 adapts two of the three approaches (the

complete search and fuzzy pattern trees; the evolutionary variant is excluded as nondeterministic and, in the source comparison, the least readable) from genre classification to difficulty grading, where to our knowledge they have not previously been applied.

2.5 Guitar Playability Metrics

For the guitar specifically, Vélez Vásquez et al. [25] proposed a playability metric for rhythm-guitar chord progressions, built from expert-informed criteria: uncommonness of chord shapes, finger displacement between chords, barre usage, right-hand strumming complexity, and rhythmic regularity. Their work is an important precedent for descriptor design — several of the descriptors developed in this thesis, such as the barre ratio, chord stretch, and shift distance, have direct analogues in their rubric — but its scope is limited to strummed chord charts in popular music. Solo classical repertoire is a substantially harder domain: textures are polyphonic and contrapuntal, the left hand moves horizontally through fretboard positions rather than between a fixed vocabulary of chord shapes, and the right hand executes arpeggiation and voice-separation patterns that have no counterpart in strumming. This thesis extends the playability-descriptor idea to that domain and connects it, for the first time, to an external professionally graded repertoire resource.

2.6 Datasets

The dataset constructed in Chapter 4 draws on several resources that play distinct roles. Difficulty labels come from GuitarBurst [8], an online graded repertoire database whose 1–20 difficulty grades serve as the prediction target. Symbolic scores come from two published research datasets: DadaGP [18], a large corpus of GuitarPro tablature transcriptions in a tokenized format, from which the classical guitar subset is used, and GAPS [17], a smaller, high-quality classical guitar dataset with MusicXML scores aligned to audio performances. The bulk of the symbolic scores, however, come from ClassClef, an online classical-guitar score library rather

than a research dataset; because it has no associated publication, it is described where the corpus is actually assembled (Chapter 4) rather than surveyed here. None of these resources alone pairs symbolic scores with difficulty grades at useful scale, which is why the matching and alignment pipeline described in Chapter 4 was necessary.

Chapter 3

Preliminaries

This chapter introduces the concepts and notation used throughout the remainder of the thesis: the technical vocabulary of classical guitar playing on which the descriptors are built, the symbolic music representations from which scores are parsed, the sequence-alignment algorithm underlying the rhythm-recovery pipeline, the ordinal classification framework and its evaluation measures, and the baseline models used for comparison.

3.1 The Classical Guitar

The classical guitar has six strings, conventionally numbered 1 to 6 from the highest-pitched to the lowest-pitched string, stretched over a fretted neck. Pressing a string against fret f raises its pitch by f semitones relative to the unfretted (*open*) string, which we denote as fret 0. A played note can therefore be identified with a pair (s, f) of string $s \in \{1, \dots, 6\}$ and fret $f \geq 0$. Because the tuning intervals between adjacent strings are smaller than the pitch range of each string, the same pitch can typically be produced at several distinct (s, f) combinations; the choice among them is a fingering decision with direct consequences for playability. Figure 3.1 illustrates the coordinate system and this pitch ambiguity.

In this thesis, a symbolic guitar score is represented as a sequence of *chord events*: the notes of the piece grouped by onset time, so that each event contains one or more simultaneously attacked notes, each carrying its string and fret. The *position*

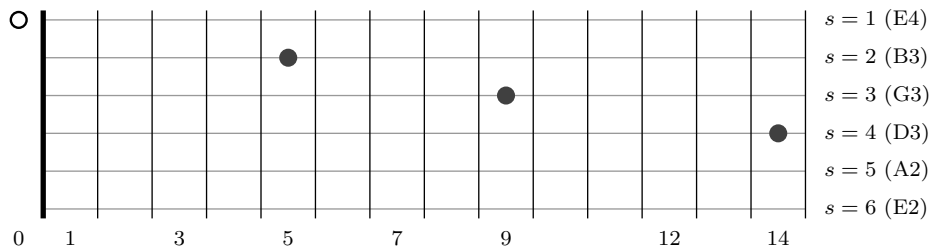


Figure 3.1: The fretboard coordinate system: strings s (with their open-string pitches in standard tuning) and frets f , with fret 0 denoting the open string. The four marked positions — $(1, 0)$ open, and $(2, 5)$, $(3, 9)$, $(4, 14)$ fretted — all produce the same pitch E4; choosing among them is a fingering decision with direct consequences for playability.

of an event refers to the mean fret of its notes, reflecting where along the neck the left hand is placed.

The technical demands of a piece divide naturally between the two hands:

- **Left hand.** The left hand stops the strings against the frets. Its principal difficulty factors are the *stretch* of a chord (the fret span the fingers must cover simultaneously), *position shifts* (movements of the entire hand along the neck between events), and *barre* chords, in which one finger is laid flat across several strings at the same fret — a fatiguing technique that is a well-known obstacle in intermediate repertoire. Open strings, by contrast, cost the left hand nothing, which is why beginner repertoire is written predominantly in the first position with heavy open-string use.
- **Right hand.** The right hand plucks the strings. Its difficulty factors include *string crossing* (successive notes on different strings), *arpeggiation* patterns that cycle the fingers across strings, and the simultaneous maintenance of multiple voices — classical guitar textures are frequently polyphonic, with a melody, bass line, and inner voices carried by one instrument.

This division, together with global properties of the score such as length, note density, and degree of polyphony, motivates the three descriptor dimensions used in Chapter 4.

3.2 Symbolic Music Representations

Symbolic music representations encode a piece as discrete note events rather than as audio. Three families of representations are relevant to this thesis.

Score-level formats encode the notated music. MusicXML [7] is an XML-based interchange format that represents pitches, durations, time signatures, voices, and notational details, and is the common target format into which all sources in this thesis are consolidated. GuitarPro is a tablature-oriented format widespread among guitarists; the DadaGP corpus [18] provides such transcriptions in a tokenized textual form. *Tablature* itself is a guitar-specific notation that specifies, for every note, the string and fret at which it is to be played — thereby fixing a fingering — in contrast to standard staff notation, which specifies only pitch.

Performance-level formats encode note events in time. MIDI represents a performance as note-on and note-off messages with associated pitches and timestamps. From a MIDI file, each note n_t has an onset time $\text{Onset}(n_t)$, and the *inter-onset interval*

$$\text{IOI}(n_t) = \text{Onset}(n_{t+1}) - \text{Onset}(n_t) \quad (3.1)$$

measures the time between successive note attacks. Inter-onset intervals are often a more reliable rhythmic signal than sounding durations: on the guitar in particular, bass strings ring far beyond their notated values, so the interval to the next attack reflects the notated rhythm more faithfully than the physically sounding length of a note.

Finally, a large share of the classical guitar repertoire is available only as *PDF scores* distributed by online tablature archives, chief among them ClassClef (described in Chapter 4, where the corpus is assembled). Vector-graphic PDFs can be parsed to recover the printed symbols; for tablature scores this yields pitch, string, and fret reliably, but the graphical encoding of note durations is not reliably recoverable, so the rhythmic dimension of such scores is lost. Recovering it is the purpose of the alignment pipeline described in Section 4.2, which builds on the algorithm introduced next.

3.3 Global Sequence Alignment

Global sequence alignment addresses the problem of finding an optimal correspondence between two sequences that may differ through local insertions, deletions, and substitutions. The Needleman–Wunsch algorithm [14], originally developed for biological sequences, solves this problem by dynamic programming. Given two sequences $A = \langle a_1, \dots, a_n \rangle$ and $B = \langle b_1, \dots, b_m \rangle$, a similarity score $s(a_i, b_j)$ for matching element a_i with b_j , and a gap penalty d for leaving an element unmatched, the algorithm fills a matrix $F \in \mathbb{R}^{(n+1) \times (m+1)}$ with $F(i, 0) = i \cdot d$, $F(0, j) = j \cdot d$, and

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(a_i, b_j) & \text{(match or substitution)} \\ F(i-1, j) + d & \text{(gap in } B) \\ F(i, j-1) + d & \text{(gap in } A). \end{cases} \quad (3.2)$$

Backtracking from $F(n, m)$ yields a globally optimal alignment. The tolerance for gaps is what makes the algorithm suitable for aligning two renditions of the same piece of music, where repeats, ornaments, and transcription discrepancies produce local insertions and deletions between otherwise corresponding note sequences.

3.4 Ordinal Classification

Ordinal classification is supervised classification into a label set $\{0, 1, \dots, K-1\}$ that carries a total order. Difficulty grading is inherently ordinal: level 12 is harder than level 8, and predicting class 2 for a class-1 piece is a far smaller error than predicting class 7. Both the model and the evaluation should respect this structure rather than treat the classes as unrelated categories.

A standard way to inject the ordering into a learning problem is the cumulative binary encoding of Cheng et al. [5]: a label $k \in \{0, \dots, K-1\}$ is represented as K binary targets

$$y_j = \begin{cases} 1 & \text{if } j \leq k \\ 0 & \text{otherwise,} \end{cases} \quad j = 0, \dots, K-1, \quad (3.3)$$

that is, a vector of ones up to and including the true class, followed by zeros. Each of the K outputs of a model trained against this encoding estimates the probability

that the instance’s label reaches at least class j , and the monotone structure of the targets encodes the class order.

Evaluating ordinal predictions likewise requires measures that account for the magnitude of errors, not only their existence. Beyond exact accuracy and balanced accuracy [4] (the mean of per-class recalls, which corrects for class imbalance), we use the mean absolute error (MAE) and mean squared error (MSE) between predicted and true class indices, and two rank correlation coefficients. Spearman’s ρ [22] is the Pearson correlation between the ranks of two variables and is used in this thesis to audit individual descriptors against the difficulty labels. Kendall’s τ [11] measures how consistently a model orders instances: for n instances, it is the normalized difference between the number of concordant and discordant pairs,

$$\tau = \frac{n_c - n_d}{\binom{n}{2}}, \quad (3.4)$$

where a pair of instances is concordant if the predicted and true labels order it the same way and discordant otherwise; in the presence of ties, adjusted variants of the coefficient are employed [12].

3.5 Baseline Models

Five model families serve as baselines in this thesis.

Ordinal regression here denotes a linear model trained against the cumulative encoding of Equation 3.3, providing a floor for what the descriptors support without any nonlinearity.

Decision trees [3] recursively partition the feature space along axis-aligned thresholds and predict a class per leaf. They are interpretable in principle, but a single tree of useful depth is high-variance and typically weak.

Random forests [2] aggregate many decision trees, each trained on a bootstrap sample of the data with random feature subsampling at every split. The ensemble is a strong general-purpose classifier on tabular data and, importantly for this thesis, can exploit arbitrary interactions between features — precisely what an additive model cannot. It therefore serves as the black-box reference point throughout Chapter 5.

Random forests also provide feature importance scores, which we use in Section 5.3 as an independent check on the additive model’s learned descriptor influences.

The remaining two baselines are *fuzzy rule-based classifiers*. Fuzzy sets [27] generalize classical sets by allowing partial membership: an element belongs to a set to a degree $\mu \in [0, 1]$ rather than strictly in or out. A *fuzzy statement* such as “fret entropy is high” then carries a degree of truth instead of a binary value, and statements can be combined with fuzzy operators (e.g. minimum for *and*, maximum for *or*). A *fuzzy rule* pairs a premise built from such statements with a class conclusion, e.g. “if fret entropy is very high then class 7,” and a rule base predicts the class whose rules attain the highest average degree of truth. Because both the premises and the decision rule are stated in natural language over named features, fuzzy rule bases are interpretable in the same sense RubricNet is: a listener or teacher can read the rule directly, without needing to trust an opaque model. Section 4.5 adapts two rule-induction methods from the fuzzy music classification work reviewed in Section 2.4 [9, 24] to the difficulty grading task: a complete search over primitive rules, and fuzzy pattern trees [10, 19, 20].

Chapter 4

Method

This chapter describes our approach in four parts, mirroring the pipeline from raw data to trained model. First, the dataset is constructed by matching symbolic scores from three heterogeneous sources against an external graded repertoire database (Section 4.1). Second, a rhythm-alignment procedure recovers note durations for the PDF-derived scores, which carry pitch but no timing (Section 4.2). Third, three successive generations of guitar-specific descriptors are engineered and audited (Section 4.3). Finally, the additive RubricNet architecture is adapted to these descriptors and trained under an ordinal objective (Section 4.4).

4.1 Dataset Construction

No existing resource pairs high-quality symbolic classical guitar scores with professional difficulty grades at a scale suitable for supervised learning. The dataset for this thesis was therefore assembled by matching symbolic scores from three separate corpora against an external graded repertoire database.

4.1.1 Difficulty Labels

Difficulty labels are taken from GuitarBurst [8], an online database of graded classical guitar repertoire in which each work is assigned an integer difficulty from 1 (beginner) to 20 (advanced concert repertoire). Scraping the database produced 2,335 graded works, which serve as the label pool for matching. As a check on label quality, a

sample of pieces at each difficulty level was manually reviewed by the author, an experienced guitarist; the assigned grades were plausible across the range, with disagreements limited to adjacent levels.

4.1.2 Symbolic Score Sources

Scores were collected from three sources with markedly different characteristics:

1. **ClassClef PDFs**: vector-graphic tablature scores obtained from ClassClef, an online classical guitar score library. This is by far the largest source, but also the most problematic: parsing the PDFs recovers pitch, string, and fret reliably, while note durations are lost (Section 4.2).
2. **DadaGP** [18]: a large corpus of GuitarPro tablature transcriptions in a tokenized format, of which only the classical guitar solo subset is relevant here.
3. **GAPS** [17]: a small, high-quality corpus of classical guitar MusicXML scores aligned with audio recordings.

4.1.3 Matching and Deduplication

Symbolic files were matched to GuitarBurst entries by fuzzy string matching on title and composer, with per-source matching procedures and a manual vetting pass over low-confidence matches. Since the same piece can appear in more than one source, cross-source duplicates were detected and removed (seven confirmed duplicates), along with one PDF whose musical content did not match its metadata. The final dataset contains **716 unique classical guitar pieces** spanning the full 1–20 difficulty range; Table 4.1 shows the source breakdown.

4.1.4 Difficulty Binning

The raw 1–20 labels are heavily imbalanced: the middle of the scale is well populated (85 pieces at level 8), whereas the rarest levels contain very few pieces (levels 17–20 have 7, 8, 5, and 3 pieces, respectively); Figure 4.1 shows the full distribution. Training directly on 20 classes was attempted and failed for precisely this reason

Table 4.1: Dataset source distribution.

Source	Pieces
ClassClef PDFs	655
DadaGP (GuitarPro)	49
GAPS (MusicXML)	12
Total	716

(Section 5.4.1). The labels are therefore grouped into 8 ordinal classes by equal-frequency binning over contiguous level ranges:

$$\mathcal{C} = \{[1-3], [4-5], [6-7], [8], [9-10], [11-12], [13-15], [16-20]\}$$

yielding class sizes between 37 and 136. The bin edges were fixed once, before any model development, and never changed thereafter.

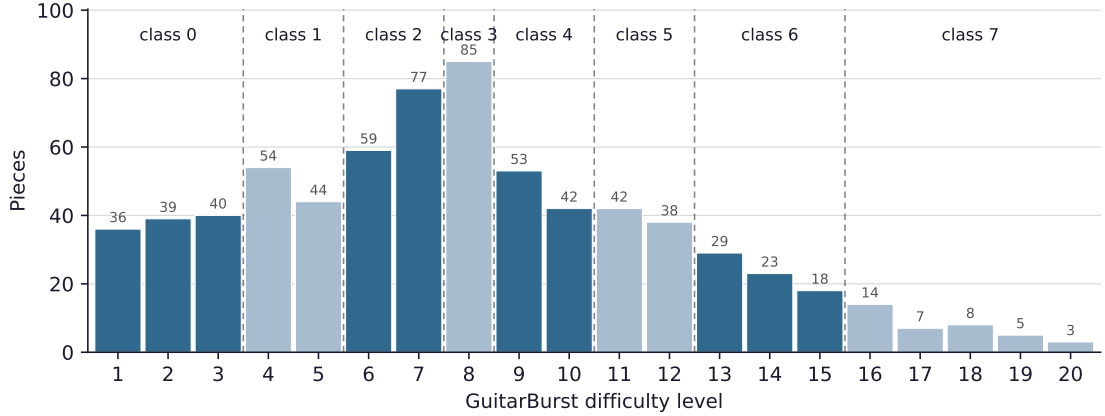


Figure 4.1: Distribution of the raw GuitarBurst difficulty levels over the 716-piece dataset. Dashed lines mark the boundaries of the 8 ordinal classes produced by equal-frequency binning; alternating shades indicate bin membership. The sparsity of the highest levels (levels 17–20 hold 7, 8, 5, and 3 pieces) is what makes direct 20-level classification untenable (Section 5.4.1).

4.1.5 Cross-Validation Splits

All experiments use a single stratified 5-fold split over the binned labels, generated once and stored alongside the dataset. Every model in this thesis — every baseline, every descriptor generation, every auxiliary experiment — is trained and evaluated on these identical folds. Freezing the splits early was a deliberate safeguard against inadvertently tuning on test data over the many experimental iterations the project went through.

4.2 Rhythm Alignment

The PDF-derived scores, 655 of the 716 pieces, carry correct pitch, string, and fret information but no usable rhythm: the vector-graphics parsing cannot recover note durations, so every note initially emerges as a uniform quarter note, and the time signature defaults to 4/4. Any descriptor involving timing — how fast a passage is, how much time a position shift has available — cannot be computed on such data.

ClassClef also provides a separate collection of roughly 6,000 MIDI performances, not paired to the PDFs in any fixed way. To recover rhythm, each PDF is first matched to a candidate MIDI from this collection, and an alignment procedure then transplants the MIDI timing onto the PDF pitch sequence in four steps:

1. **Pitch-sequence alignment.** The PDF’s note sequence and the MIDI’s note sequence are aligned with the Needleman–Wunsch algorithm (Section 3.3), whose tolerance for local insertions and deletions absorbs repeats, ornaments, and transcription discrepancies between the two renditions. Pieces whose best alignment covers too little of the score are rejected rather than force-matched.
2. **Time signature.** The time signature is taken from the MIDI meta-events rather than from the PDF parser, whose 4/4 default is frequently wrong.
3. **Duration from inter-onset intervals.** A note’s duration is set to the inter-onset interval to the next note (Equation 3.1) rather than to its sounding duration in the MIDI. As discussed in Section 3.2, guitar bass strings ring far

past their notated values, so sounding durations systematically overstate how long notes are held; onsets are the reliable signal.

4. **Serialization.** The aligned pitches with corrected timing are written back out as MusicXML.

Figure 4.2 illustrates the procedure on a short excerpt. The procedure aligned 579 of the 655 PDF pieces; the remaining 76 could not be used — nearly all rejected for low alignment coverage, three because no matching MIDI file existed. Together with the sources that natively carry rhythm (all 12 GAPS scores and 48 of the 49 DadaGP transcriptions), 639 of the 716 pieces (89.2%) end up with complete note durations, flagged per piece with a rhythm-availability indicator. After alignment, all sources were consolidated into a single MusicXML corpus so that one parser handles every piece identically, regardless of provenance.

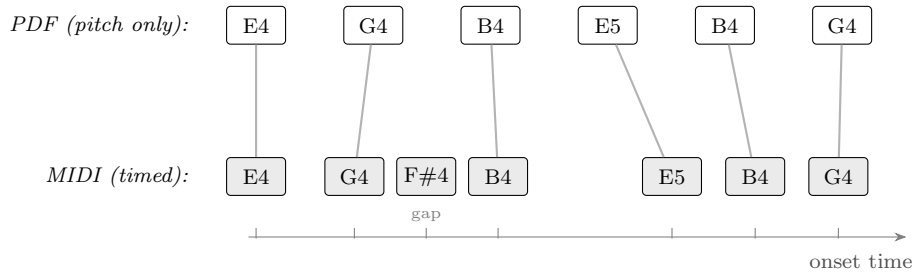


Figure 4.2: Rhythm alignment, schematically. The PDF-derived score contributes the pitch sequence but no timing (top); the matched MIDI performance contributes onset times (bottom). Needleman–Wunsch alignment pairs the two note sequences, absorbing local insertions such as the ornamental F#4 as gaps, and each PDF note receives the inter-onset interval of its MIDI counterpart as its duration.

One limitation is accepted deliberately: the aligned output collapses the score into a single voice. Classical guitar notation typically separates a treble melody from a bass line, and in the single-voice output a bass note’s duration ends at the next onset in *any* voice rather than at its notated value. For descriptor extraction this is acceptable — pitch content and attack positions, which the descriptors actually consume, are correct — but it is a genuine constraint, discussed further in Section 6.1.

4.3 Descriptor Engineering

RubricNet is strictly additive: no descriptor can interact with another before the final summation. The model cannot, for instance, learn on its own that a wide stretch is harder when it must be executed quickly — if that interaction matters, it must be built into a descriptor by hand. Conversely, an uninformative descriptor is not merely useless to an additive model: since every input is scored and summed into the output, it injects noise directly into the prediction. Descriptor design is therefore where most of the modeling effort of this thesis was invested, across three generations, each followed by a systematic audit of every descriptor’s Spearman correlation (Section 3.4) with the raw 1–20 difficulty labels.

Throughout, the input representation is the sequence of chord events introduced in Section 3.1: notes grouped by onset, each note carrying its string (1–6) and fret (0 for open strings), with “position” referring to the mean fret of an event’s notes.

4.3.1 V1: Initial Descriptor Set

The first generation contained 12 descriptors covering the most immediate candidates in each dimension:

- **Left hand:** `barre_ratio`, the fraction of chord events detected as barre chords — a chord counts as a barre when at least three of its notes sit on the same fret across three consecutive strings; `avg_chord_stretch` and `max_chord_stretch`, the mean and maximum fret span among fretted notes of multi-note chords; `avg_position_shift`, the mean absolute change in position between successive events; and `fret_change_rate`, how often the active fret set changes, normalized by note count.
- **Right hand:** `arpeggio_density`, the fraction of single-note-to-single-note transitions that change string; `avg_string_jump` and `max_string_jump`, the mean and maximum string distance between successive events; and `special_technique_ratio`, the fraction of notes marked with slurs or slides.
- **Global:** `avg_polyphony` (mean notes per event), `total_notes`, and `tempo_`

bpm.

The correlation audit of these descriptors against the raw 1–20 labels revealed substantial weaknesses. Several descriptors carried essentially no signal: `fret_change_rate` ($\rho = -0.04$), `avg_string_jump` ($\rho = -0.06$), `arpeggio_density` ($\rho = -0.10$), and `avg_polyphony` ($\rho = +0.04$), while `tempo_bpm` was both weak ($\rho = -0.16$) and missing for 14 pieces. `special_technique_ratio` proved to be nonzero for only 29 of 716 pieces — an artifact of which source formats happen to encode slur markings, not a property of the music — and was removed outright. For an additive model that must assign every input its own score and sum them all, such dead inputs are noise injected directly into the output.

4.3.2 V2: Audited and Expanded Set

The second generation addressed the problem from both ends: repairing extraction defects in the PDF parsing path that had been corrupting fret values, and adding 15 candidate descriptors designed around what the V1 audit suggested was missing — fretboard geography and movement statistics:

- **Length scaling:** `log_total_notes` = $\log(1 + \text{total_notes})$, compressing the long tail of piece lengths.
- **Fretboard position:** `avg_fret`; `p90_fret` (90th percentile of frets used); `high_position_ratio`, the fraction of notes at fret 7 or above; and `open_string_ratio`, the fraction of open-string notes — expected to correlate *negatively* with difficulty, since open strings cost the left hand nothing.
- **Stretch and shape distribution:** `p90_chord_stretch`; `chord_ratio`, the fraction of events with two or more notes; `avg_string_span`, the mean string span of chords; and `unique_shape_rate`, the fraction of distinct chord shapes among all chords played.
- **Movement:** `shift_rate`, the fraction of successive-event transitions that move the position by more than 2 frets; `max_position_shift`; and `std_position_shift`.

- **Variety:** `fret_entropy` and `string_entropy`, the Shannon entropies [21] of the fret and string usage distributions, capturing how much of the fretboard a piece actually visits; and `repetition_ratio`, the fraction of events that exactly repeat the previous chord shape.

Every candidate was audited in the same manner as V1. The position and variety descriptors proved to supply exactly the missing signal: `fret_entropy` reached $\rho = +0.65$, `high_position_ratio` $+0.56$, `avg_fret` $+0.55$, and `open_string_ratio` -0.48 with the expected negative sign. Two candidates fell below the $|\rho| < 0.05$ cutoff (`avg_string_span`, `unique_shape_rate`) and were dropped alongside `special_technique_ratio`. The surviving V2 set contains **24 descriptors**: 14 left-hand, 5 right-hand, 5 global. The remaining weak-but-nonzero V1 descriptors were deliberately retained; whether removing them helps is tested empirically in Section 5.4.2.

4.3.3 V3: Rhythm-Aware Descriptors

With the alignment pipeline delivering rhythm for 89.2% of the dataset and all sources consolidated into one MusicXML corpus, the third generation adds the descriptors that timing makes possible. A rhythm-aware parser computes an onset time in *beats* for every chord event; all V3 descriptors are expressed in beats rather than seconds, which makes them independent of tempo markings — a deliberate choice, since `tempo_bpm` had already tested as one of the weakest descriptors and is missing or unreliable for many scores.

Eight descriptors are added, in two families:

1. **Windowed bottleneck descriptors.** A piece’s difficulty is determined by its hardest passage, not its average — a mostly easy piece with four bars of demanding shifts is a hard piece. A window of 16 beats (four measures in 4/4) slides across the score, anchored at each onset, and records:
 - `max_note_density_window`: the peak notes-per-beat over any window;
 - `max_avg_chord_stretch_window`: the largest window-averaged chord stretch;

- `p95_position_shift_window`: the largest per-window 95th-percentile position shift.

2. **Context-aware physical proxies.** These divide a physical demand by the time available to execute it:

- `avg_stretch_velocity_beats` and `p90_stretch_velocity_beats`: chord stretch divided by the inter-onset gap in beats (mean and 90th percentile);
- `avg_position_shift_speed_beats` and `max_position_shift_speed_beats`: position-shift distance divided by the inter-onset gap;
- `polyphonic_arpeggio_intensity_beats`: overall note density (notes per beat) scaled by average polyphony, capturing the right hand’s workload when multiple voices must be sustained.

The intuition is pedagogical: a five-fret jump during a fermata is trivial, the same jump inside a sixteenth-note run is not, and a context-free `avg_position_shift` cannot distinguish the two.

For the 77 pieces (10.8%) without rhythm data, the eight V3 descriptors are imputed with the training-fold median, computed separately inside each cross-validation fold to avoid leakage. Before adopting the new extraction path, gate experiments confirmed that the switch to the unified corpus did not regress the V2 base descriptors.

The final V3 set contains **32 descriptors**: 20 left-hand, 6 right-hand, 6 global.

4.4 Model Architecture and Training

RubricNet [15] is an additive ordinal neural network. Let $x \in \mathbb{R}^M$ be the vector of M standardized descriptors for a piece ($M = 32$ for V3). Figure 4.3 shows the complete architecture.

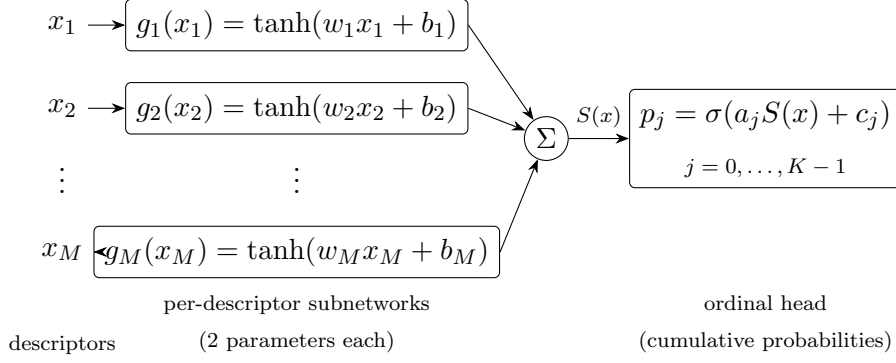


Figure 4.3: The RubricNet architecture. Each descriptor x_i is scored independently by a two-parameter subnetwork g_i ; the bounded scores are summed into the aggregated difficulty score $S(x)$, and an ordinal head maps $S(x)$ to K cumulative class probabilities. No descriptor interacts with any other before the summation, so $g_i(x_i)$ is exactly descriptor i ’s contribution to the prediction.

4.4.1 Additive Subnetworks

Each descriptor x_i is scored by its own subnetwork $g_i : \mathbb{R} \rightarrow (-1, 1)$, consisting of a single affine transformation followed by a tanh nonlinearity:

$$g_i(x_i) = \tanh(w_i x_i + b_i), \quad (4.1)$$

with scalar parameters w_i, b_i . The bounded output ensures that no single descriptor can dominate the prediction: each contributes at most one unit of score in either direction. The aggregated difficulty score is the plain sum

$$S(x) = \sum_{i=1}^M g_i(x_i). \quad (4.2)$$

This constitutes the entire representational budget of the model — two parameters per descriptor and no hidden interactions — and it is what makes the qualitative analysis of Section 5.3 possible: $g_i(x_i)$ is descriptor i ’s contribution, exactly, not an approximation obtained by a post-hoc attribution method.

4.4.2 Ordinal Prediction Head

Difficulty classes are ordered, and the model should be penalized more for confusing class 1 with class 7 than with class 2. Labels are therefore represented with the

cumulative binary encoding of Equation 3.3. The model maps the aggregated score through a final linear layer with K outputs and a sigmoid,

$$p_j = \sigma(a_j S(x) + c_j), \quad (4.3)$$

so that each p_j estimates the probability that the piece’s difficulty reaches at least class j . At inference time, the predicted class is the number of consecutive outputs from $j = 0$ upward that exceed 0.5, minus one.

Training minimizes the squared error between the predicted and target cumulative vectors,

$$\mathcal{L} = \sum_{j=0}^{K-1} \omega_j (p_j - y_j)^2, \quad (4.4)$$

where ω_j are inverse-frequency class weights computed on the training fold, compensating for the residual class imbalance that equal-frequency binning cannot fully remove.

4.4.3 Ordinal Label Smoothing

Equal-frequency bins draw hard boundaries through what is in reality a continuum: a piece at GuitarBurst level 10 sits at the edge of its bin, and misclassifying it into the adjacent bin is a much smaller error than the hard targets acknowledge. To soften this, the step in the cumulative target can optionally be replaced by a sigmoid ramp,

$$y_j^{\text{smooth}} = \sigma\left(\frac{k - j + 0.5}{T}\right), \quad (4.5)$$

with temperature T ; as $T \rightarrow 0$ this recovers the hard encoding exactly. The effect of T is evaluated in Section 5.4.3; the headline results use the hard targets.

4.4.4 Training Procedure and Hyperparameter Search

Descriptors are standardized with a scaler fit on the training fold only. The model is trained with early stopping on validation accuracy (patience of 20 epochs). Hyperparameters were tuned per descriptor generation with Optuna [1], searching the learning rate (5×10^{-3} – 10^{-1} , log scale), batch size (16–64), dropout [23] on the input descriptors (0.1–0.5), weight decay (10^{-4} – 10^{-1} , log scale), and learning-rate

decay (0.3–0.9), with validation balanced accuracy as the objective. The selected V3 configuration uses batch size 64, learning rate 0.047, dropout 0.20, weight decay 0.064, and learning-rate decay 0.42. Notably, the architecture itself was never part of the search space: the one-scalar-per-descriptor structure is the point of the model, not a tunable capacity parameter.

Final results are averaged over 3 random seeds $\times$ 5 folds (15 runs) for RubricNet; the deterministic baselines are averaged over the 5 folds.

4.5 Fuzzy Rule-Based Comparison Methods

Two additional interpretable baselines are adapted from the fuzzy rule-based classifiers compared by Heerde, Vatolkin, and Rudolph for music genre recognition [9]: a complete search over primitive rules [24] and fuzzy pattern trees [10, 19, 20]. Both operate on the V3 descriptor set, are fully deterministic given the training data, and are therefore evaluated once per fold rather than over multiple seeds. Following the source paper, both treat the eight difficulty classes as nominal and predict by one-vs-all argmax; the ordinal structure of the task is not injected into either method, which lets the comparison in Chapter 5 isolate what is gained by RubricNet’s ordinal design specifically.

4.5.1 Fuzzification

Both methods require every descriptor to be expressed as degrees of truth for five linguistic terms: very low, low, moderate, high, and very high. Raw descriptor values are first mapped to $[0, 1]$ and then assigned to five evenly spaced triangular membership functions centered at 0, 0.25, 0.5, 0.75, 1 (Figure 4.4), so that adjacent terms always sum to a membership of exactly 1.

Because several V3 descriptors are heavy-tailed (e.g. total note count, window maxima), a plain min-max normalization would compress most pieces into the lowest term. Instead, each descriptor is normalized by its empirical cumulative distribution function (CDF) on the training fold: a value is mapped to the fraction of training pieces at or below it, so that “moderate” corresponds to the corpus median regardless

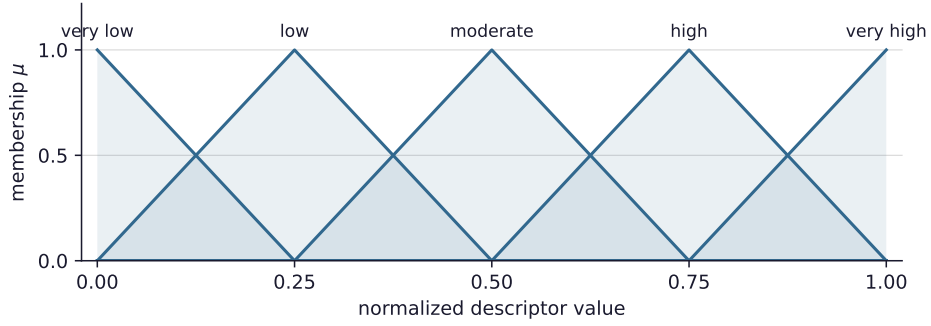


Figure 4.4: The five triangular membership functions over the normalized descriptor range. Each normalized value belongs to at most two adjacent linguistic terms, with memberships summing to 1.

of the descriptor’s raw scale. Values in the validation and test folds are clipped to $[0, 1]$ before fuzzification. A plain min-max variant is evaluated as an ablation in Section 5.4.

4.5.2 Complete Search of Primitive Rules

A primitive rule has the form “if descriptor X is term T then class C .” For every descriptor–term pair and class, the relevance of the corresponding rule is estimated on the training fold as

$$R(C, X, T) = P(X \text{ is } T \mid C) \cdot (1 - P(X \text{ is } T)), \quad (4.6)$$

where the fuzzy probabilities are estimated as mean memberships over the corresponding pieces [24]. The first factor rewards terms that are typical of class C ; the second penalizes terms that are typical of the corpus as a whole, so that a term satisfied by almost every piece (uninformative) scores low even if it happens to hold for most of class C . For each class, the 32 descriptors $\times$ 5 terms are ranked by relevance, and a prediction is made by averaging the truth of the top- m rules per class and taking the argmax over classes. The number of rules m is selected per fold on the validation split by balanced accuracy, then the rule ranking is refit on the combined training and validation data before scoring the test fold.

4.5.3 Fuzzy Pattern Trees

A fuzzy pattern tree (FPT) represents one class as a single tree whose leaves are fuzzy statements and whose inner nodes are fuzzy operators — here, *and* (minimum), *or* (maximum), and *avg* (mean of the two children), following [19]. Leaves may additionally be negated (degree of truth $1 - \mu$), doubling the candidate statement pool to 320.

Induction is deterministic and top-down, one tree per class. The tree is initialized as the single statement minimizing the root-mean-squared error (RMSE) against the one-vs-all binary target on the training fold. At each step, every current leaf is considered for replacement by a new operator node whose two children are the old leaf and a new candidate statement; every (leaf, operator, statement) combination that does not push the tree beyond the maximum depth $d_{\max}$ is evaluated, and the extension with the lowest resulting RMSE is applied. Induction stops when no legal extension remains, the depth limit is reached on every branch, or the best available extension would worsen the error by more than a factor $(1 + \gamma)$, with $\gamma = 0.05$. $d_{\max}$ is swept over $\{1, \dots, 5\}$ and selected per fold on the validation split by balanced accuracy, mirroring the complete search’s selection of m ; $d_{\max} = 1$ yields a primitive, single-statement tree.

Because each one-vs-all target is a minority class (5–19% positive across the eight classes), unweighted RMSE is minimized by trees that predict close to zero everywhere. Induction therefore uses a class-balanced RMSE by default, weighting positive and negative instances so that each contributes equally regardless of class size; a constant predictor then achieves its best possible (weighted) error at an output of 0.5, rather than at the majority class. The unweighted variant is evaluated as an ablation in Section 5.4, alongside a variant without negated leaves.

Chapter 5

Evaluation

In this evaluation, we assess the effectiveness of the proposed approach on the 716-piece dataset described in Chapter 4. The evaluation comprises both quantitative and qualitative analyses: the former compares RubricNet against baseline models across the three descriptor generations, tests the multi-dimensional descriptor design through an ablation, and reports a coarse three-level view of the results; the latter examines whether the trained model’s internals in fact read as a difficulty rubric. The chapter closes with a series of auxiliary experiments — most of them negative results — that shaped the final design.

5.1 Evaluation Setup

All experiments run on the fixed stratified 5-fold split described in Section 4.1. RubricNet results are averaged over 3 random seeds $\times$ 5 folds (15 runs); the deterministic baselines are averaged over the 5 folds. Six metrics are reported (see Section 3.4 for definitions):

1. **Accuracy**: the fraction of exactly correct difficulty bins.
2. **Balanced accuracy**: the mean per-class recall, correcting for the residual class imbalance (class sizes range from 37 to 136).
3. **Accuracy $\pm$ 1**: the fraction of predictions within one bin of the true class.
4. **Mean absolute error (MAE)** in bin indices.

5. **Mean squared error (MSE)** in bin indices, which penalizes distant misses more heavily.
6. **Kendall’s τ** : the rank correlation between predicted and true classes, measuring how well the model orders pieces by difficulty.

For an ordinal task, the error metrics matter as much as raw accuracy: a model that misses by one bin is far more useful to a teacher than one that misses by four, and accuracy alone cannot distinguish between them.

Five baselines (Section 3.5) are compared against RubricNet on every descriptor generation: an ordinal regression model using the same cumulative target encoding, a decision tree (depth 6), a Random Forest (200 trees), and two fuzzy rule-based classifiers evaluated only on V3 (Section 4.5). The Random Forest is the strongest of the tree-based baselines and serves as the black-box reference point throughout: it can exploit arbitrary feature interactions, which the additive RubricNet by construction cannot. The fuzzy classifiers add a second interpretable reference point that, unlike RubricNet, makes no use of the ordinal class structure.

5.2 Quantitative Analysis

5.2.1 8-Class Difficulty Estimation

Table 5.1 shows the full comparison across the three descriptor generations.

Four observations stand out from the table.

First, **descriptor quality, not model choice, drove the improvement**. RubricNet’s architecture never changed between V1 and V3, yet its accuracy climbed from 0.243 to 0.310 purely through better inputs. On V1, RubricNet trailed the Random Forest by more than seven accuracy points — unsurprising, since half of the V1 descriptors carried almost no signal, and an additive model must sum in every input it is given, noise included. A Random Forest can simply route around uninformative features; RubricNet cannot.

Second, **on the metrics that matter for ordinal classification, the interpretable model outperforms the black box**. RubricNet V2 already overtakes

Table 5.1: 8-class difficulty estimation across descriptor generations (mean $\pm$ std; RubricNet V2/V3 over 3 seeds $\times$ 5 folds, RubricNet V1 over a single seed, baselines over 5 folds). Bold marks the best value per generation.

Model	Accuracy	Balanced Acc.	Acc. $\pm$ 1	MAE	MSE	Kendall τ
Ordinal Regression V1	0.214 $\pm$ 0.062	0.197 $\pm$ 0.051	—	1.447 $\pm$ 0.260	3.431 $\pm$ 0.907	—
Decision Tree V1	0.282 $\pm$ 0.030	0.255 $\pm$ 0.033	—	1.352 $\pm$ 0.086	3.473 $\pm$ 0.338	—
Random Forest V1	0.316 $\pm$ 0.041	0.278 $\pm$ 0.026	—	1.222 $\pm$ 0.081	2.951 $\pm$ 0.223	—
RubricNet V1	0.243 $\pm$ 0.042	0.231 $\pm$ 0.031	—	1.258 $\pm$ 0.092	2.820 $\pm$ 0.380	—
Ordinal Regression V2	0.218 $\pm$ 0.056	0.214 $\pm$ 0.056	—	1.316 $\pm$ 0.339	2.906 $\pm$ 1.172	—
Decision Tree V2	0.286 $\pm$ 0.036	0.248 $\pm$ 0.031	—	1.295 $\pm$ 0.118	3.127 $\pm$ 0.550	0.510 $\pm$ 0.058
Random Forest V2	0.320 $\pm$ 0.029	0.275 $\pm$ 0.025	—	1.116 $\pm$ 0.047	2.487 $\pm$ 0.202	0.609 $\pm$ 0.022
RubricNet V2	0.300 $\pm$ 0.032	0.283 $\pm$ 0.033	0.730 $\pm$ 0.052	1.115 $\pm$ 0.102	2.372 $\pm$ 0.445	0.623 $\pm$ 0.038
Ordinal Regression V3	0.309 $\pm$ 0.033	0.288 $\pm$ 0.033	—	1.109 $\pm$ 0.143	2.467 $\pm$ 0.762	—
Decision Tree V3	0.279 $\pm$ 0.036	0.242 $\pm$ 0.034	—	1.286 $\pm$ 0.059	3.130 $\pm$ 0.491	0.518 $\pm$ 0.040
Random Forest V3	0.331 $\pm$ 0.022	0.289 $\pm$ 0.018	—	1.122 $\pm$ 0.071	2.527 $\pm$ 0.312	0.599 $\pm$ 0.042
Fuzzy Rules (CS) V3	0.264 $\pm$ 0.044	0.236 $\pm$ 0.044	0.617 $\pm$ 0.068	1.433 $\pm$ 0.168	3.803 $\pm$ 0.597	0.464 $\pm$ 0.075
Fuzzy Pattern Tree V3	0.268 $\pm$ 0.047	0.252 $\pm$ 0.041	0.623 $\pm$ 0.062	1.469 $\pm$ 0.207	4.233 $\pm$ 0.951	0.484 $\pm$ 0.063
RubricNet V3	0.310 $\pm$ 0.027	0.291 $\pm$ 0.028	0.733 $\pm$ 0.032	1.069 $\pm$ 0.063	2.147 $\pm$ 0.255	0.634 $\pm$ 0.026

the Random Forest on balanced accuracy, MAE, MSE, and Kendall’s τ , and V3 extends all four leads: balanced accuracy (0.291 vs. 0.289), MAE (1.069 vs. 1.122), MSE (2.147 vs. 2.527), and τ (0.634 vs. 0.599). The Random Forest retains a two-point advantage in raw exact accuracy (0.331 vs. 0.310), but the error metrics reveal the decisive trade-off: when RubricNet errs, it errs by less (-0.053 MAE), and it preserves the difficulty ordering more faithfully (τ gain of $+0.035$). For an ordinal task in which a one-bin miss is far preferable to a four-bin miss, these error and ranking metrics are the appropriate lens. Given that the entire purpose of the additive constraint is interpretability, trading two points of raw accuracy for better error distances, superior ranking correlation, and complete transparency is a favorable exchange.

Third, **the V3 rhythm descriptors help RubricNet more than they help the baselines**. The Random Forest gains almost nothing from V2 to V3 (accuracy 0.320 to 0.331 with worse MAE and MSE), while RubricNet improves on every metric. This is consistent with the design intent: the context-aware descriptors hand-encode exactly the demand-versus-time interactions that the additive model cannot learn on its own, whereas the forest could already approximate such interactions from the

raw V2 inputs.

The error structure behind these numbers is visible in Figure 5.1, which pools the RubricNet V3 test predictions of one seed across all five folds. The mass concentrates on the diagonal and its immediate neighbors — 73% of predictions fall within one bin of the truth — and predictions far from the diagonal are rare, which is precisely the behavior the MAE and Kendall’s τ figures summarize. The matrix also shows where the model still struggles: the easiest pieces (class 0) are most often placed one bin too high, and the middle classes blur into their neighbors, consistent with equal-frequency bins drawing hard boundaries through a continuum.

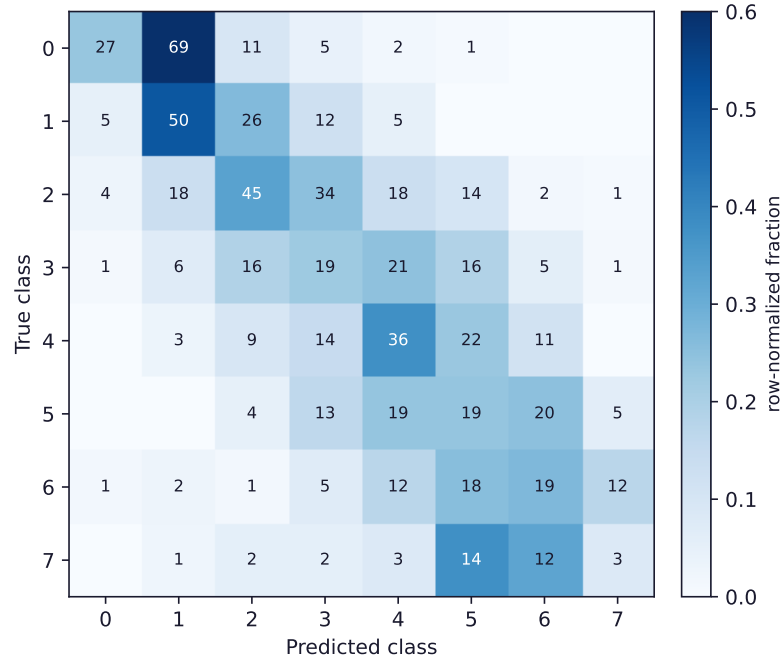


Figure 5.1: Confusion matrix of RubricNet V3 test predictions (seed 0, pooled over the five test folds; cell counts annotated, rows normalized by true-class size). Errors concentrate within one bin of the diagonal.

Fourth, **both fuzzy rule-based classifiers trail every other baseline, and the gap is explained by their nominal treatment of an ordinal task rather than by weak rule quality.** Complete search and fuzzy pattern trees land close to each other (accuracy 0.264 and 0.268) and comfortably above the 0.125 random floor, but well below the decision tree (0.279) and Random Forest (0.331). The qualitative

rules in Section 5.3 show the induced premises are sensible — low fretboard position and stretch for the easiest class, their opposites for the hardest — so the shortfall is not that the rules fail to capture difficulty-relevant structure. Rather, both methods predict each class independently via one-vs-all argmax, with no mechanism preventing a confident but wrong prediction from landing far from the true class; this shows up most clearly in Kendall’s τ (0.464 and 0.484), which trails RubricNet V3’s 0.634 by a wide margin despite the fuzzy methods using the same descriptors. RubricNet, in contrast, is both interpretable and ordinal by construction, and the comparison isolates the value of that combination: interpretability alone, without an ordinal decision rule, leaves accuracy and ranking quality on the table.

5.2.2 Per-Dimension Ablation

The third research question asks whether combining left-hand, right-hand, and global descriptors outperforms any single dimension. To isolate the feature groups from RubricNet’s training variance, the ablation uses the Random Forest under the identical evaluation protocol (same folds, same imputation), trained on each V3 dimension separately and on the full set. Table 5.2 shows the result.

Table 5.2: Per-dimension ablation with Random Forest on V3 descriptors (mean $\pm$ std over 5 folds).

Feature set	#	Accuracy	Balanced Acc.	MAE	MSE
Left hand only	20	0.307 $\pm$ 0.049	0.261 $\pm$ 0.049	1.272 $\pm$ 0.066	3.158 $\pm$ 0.309
Right hand only	6	0.229 $\pm$ 0.033	0.203 $\pm$ 0.031	1.697 $\pm$ 0.061	5.171 $\pm$ 0.422
Global only	6	0.283 $\pm$ 0.021	0.261 $\pm$ 0.016	1.374 $\pm$ 0.081	3.579 $\pm$ 0.371
All dimensions	32	0.331 $\pm$ 0.022	0.289 $\pm$ 0.018	1.122 $\pm$ 0.071	2.527 $\pm$ 0.312

The combination wins on every metric. The left hand is the strongest single dimension — as expected for classical guitar, where stretches, shifts, and barre work dominate technical difficulty — but even it falls 2.4 accuracy points and 0.15 MAE short of the full set. The right-hand dimension is weak in isolation (0.229 accuracy from its 6 descriptors) yet still contributes: the gap between left-hand-only and all-dimensions shows that dropping it would leave signal unused. The global

dimension packs considerable signal into 6 descriptors, driven mostly by piece length.

5.2.3 Coarse 3-Class Evaluation

Much prior work grades difficulty on a three-level scale, so for comparability the 8-class predictions are mapped at evaluation time onto Easy (classes 0–2), Medium (3–5), and Hard (6–7). Table 5.3 summarizes the outcome.

Table 5.3: Coarse 3-class evaluation (Easy / Medium / Hard), obtained by mapping 8-class predictions.

Model	Accuracy	Balanced Accuracy
RubricNet V2	0.670 ± 0.039	0.621 ± 0.046
RubricNet V3	0.676 ± 0.033	0.614 ± 0.052

At this granularity, RubricNet V3 places two out of three pieces in the correct broad category. For the practical use case — filtering a repertoire database down to a set of candidates that is roughly appropriate for a student before a teacher makes the final selection — this level of accuracy is already usable.

5.3 Qualitative Analysis

The quantitative results establish that the additive model is competitive with a black-box baseline. This section examines the property that motivates the additive constraint in the first place: whether the trained model’s internals actually read as a difficulty rubric. We analyze which descriptors carry the predictions, verify that their contributions behave monotonically across difficulty classes, and evaluate the context-aware descriptor design directly.

5.3.1 Descriptor Influence

Because every descriptor’s contribution is the scalar $g_i(x_i)$, the influence of descriptor i over a test set $\mathcal{D}$ can be measured directly as the range its subnetwork actually

uses:

$$\text{Influence}(i) = \max_{x \in \mathcal{D}} g_i(x_i) - \min_{x \in \mathcal{D}} g_i(x_i). \quad (5.1)$$

A descriptor whose output barely moves across the test set cannot move the aggregated score $S(x)$ across class thresholds; one that sweeps most of its $(-1, 1)$ output range is doing substantive work. All analyses below use fold 0 of seed 0.

In the V3 model, the five most influential descriptors are `log_total_notes` (range 1.93), `total_notes` (1.93), `shift_rate` (1.91), `open_string_ratio` (1.85), and `string_entropy` (1.83). This reads as a plausible difficulty rubric. Piece length dominates — longer pieces demand more sustained concentration and simply contain more opportunities for difficult material, and graded repertoire lists reflect this. The frequency of left-hand position shifts (`shift_rate`) is the top technical descriptor, matching the pedagogical reality that moving cleanly along the neck is a central obstacle in intermediate repertoire. `open_string_ratio` works in the opposite direction: its subnetwork learned a negative slope, so a high share of open strings pulls the predicted difficulty down, exactly as a teacher would expect from beginner-friendly first-position writing. `string_entropy` rewards pieces that spread the right hand’s work across many strings.

For comparison, the V2 model’s top five were `log_total_notes` (1.99), `avg_string_jump` (1.97), `total_notes` (1.94), `fret_entropy` (1.90), and `chord_ratio` (1.74). The global length signal is stable across generations, while the technical emphasis shifted between V2 and V3 from fretboard-coverage variety toward explicit movement descriptors once the rhythm-aware inputs became available.

A legitimate concern about any learned “importance” is that it may reflect training noise rather than signal. Figure 5.2 addresses this by triangulation: RubricNet’s influence ranges are compared against two independent measures — Random Forest feature importances and absolute Spearman correlations with the raw labels. The three paradigms broadly agree on which descriptors matter: `total_notes`, the fret-coverage and position descriptors, and the movement statistics rank highly under all three, while the descriptors the audit flagged as weak remain near the bottom. The agreement across an additive neural model, an ensemble of trees, and a model-free rank statistic makes it unlikely that RubricNet’s learned emphasis is an artifact of

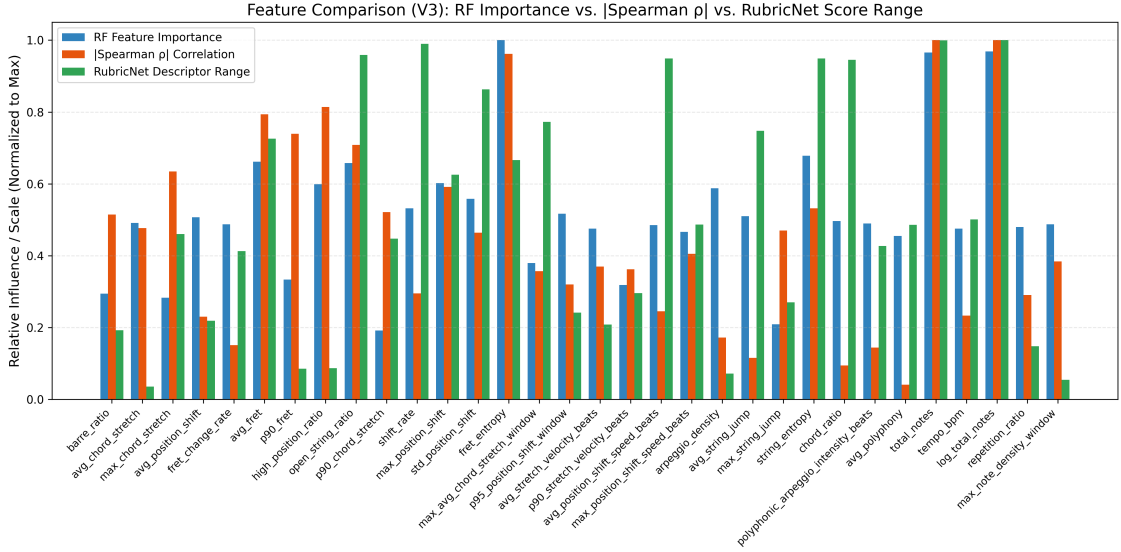


Figure 5.2: Descriptor importance under three independent measures: RubricNet influence ranges, Random Forest feature importances, and absolute Spearman correlations with the raw difficulty labels (V3 descriptors, fold 0).

its own training.

5.3.2 Monotonicity

For the rubric reading to be trustworthy, descriptor contributions should behave monotonically: pieces in higher difficulty classes should, on average, receive higher scores from difficulty-increasing descriptors. In an unconstrained network there is no reason for this to hold. In RubricNet, each g_i is a monotone function of its input by construction (an affine map through tanh), but monotonicity *across difficulty classes* is an empirical property of the data and training, not a guarantee.

Figure 5.3 plots the mean subnetwork output per true difficulty class. The influential descriptors show steady trends from class 0 through class 7 — rising for the difficulty-increasing descriptors, falling for `open_string_ratio` — while the low-influence descriptors stay flat or fluctuate within a narrow band, consistent with their small learned ranges: they vary, but they barely move the sum. This is the property that makes the model’s explanations usable in practice: if a piece is graded class 5 rather than class 3, the difference decomposes into specific descriptor

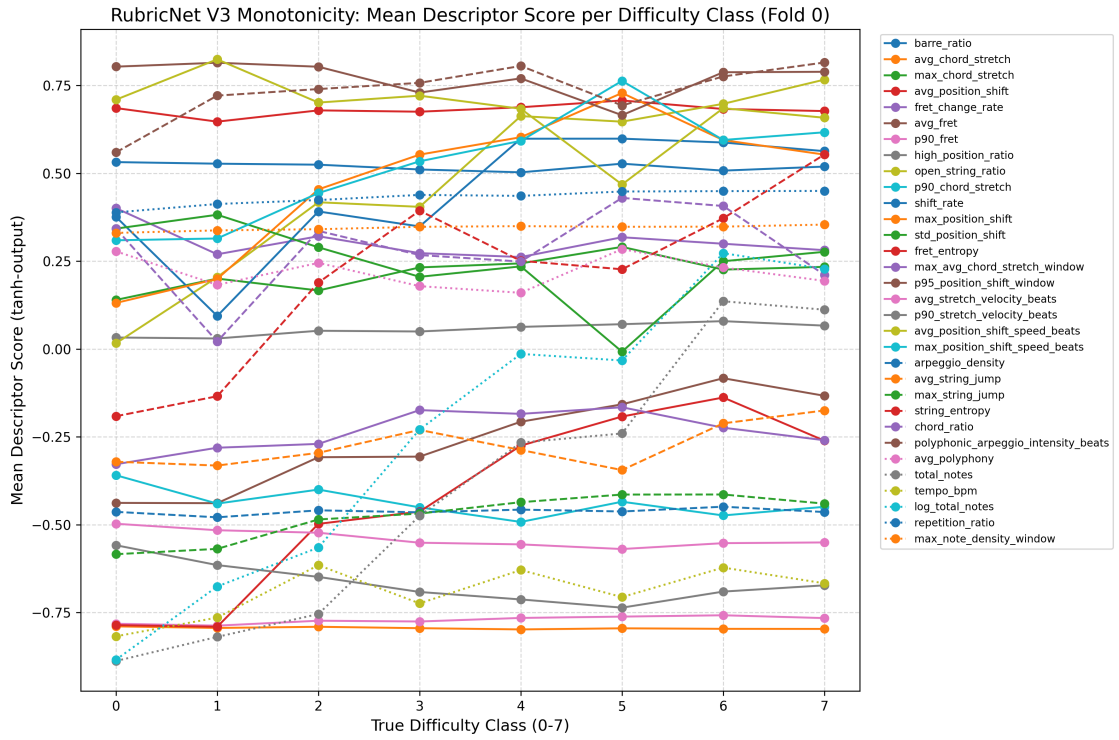


Figure 5.3: Mean per-descriptor subnetwork output by true difficulty class (V3 model, fold 0). Contributions rise steadily with difficulty for the influential descriptors; `open_string_ratio` falls, consistent with open strings making music easier to play.

contributions — more frequent shifts, higher positions, denser texture — each of which moves in a direction a teacher can verify against the score.

Figure 5.4 makes this concrete with two test pieces from fold 0 at opposite ends of the scale: Sor’s Andantino Op. 35 no. 2, a canonical beginner study (GuitarBurst level 1), and Bach’s Chaconne from the second violin partita (level 19), a summit of the concert repertoire. The model places the Andantino in class 0 (correct) and the Chaconne in class 6 (true class 7), and the score sheets behind the two predictions read exactly as a teacher would expect: the Chaconne is pushed upward by its sheer length, sustained position-shift activity, and dense multi-voice texture, while the Andantino is pulled downward by its heavy open-string use, low fretboard positions, and narrow fretboard coverage. Notably, both pieces receive similar *negative* contributions from the stretch descriptors — wide chord stretches are simply not what distinguishes these two works — illustrating that the rubric does not inflate every descriptor for a hard piece, but attributes the difficulty to the specific demands the score actually makes.

5.3.3 Context-Free vs. Context-Aware Descriptors

The V3 design rests on the hypothesis that physical difficulty depends on the time available to execute a demand. Table 5.4 tests this hypothesis directly, comparing Spearman correlations with the raw 1–20 labels for context-free descriptors and their context-aware counterparts on the 639-piece rhythm subset.

Every context-aware descriptor outperforms its context-free source, and the position-shift family shows the starkest gap: raw shift distance is nearly uninformative ($\rho = +0.13$), because a large shift with ample time is not difficult — easy pieces abound with leisurely jumps between phrases. Dividing the same quantity by the inter-onset time in beats more than triples the correlation (+0.44 for the peak variant). The arpeggio case is the most instructive: `arpeggio_density` on its own correlates *negatively* with difficulty, since constant string crossing is characteristic of easy arpeggio studies; only when scaled by polyphony and note rate does the descriptor recover the intended signal, namely that crossing strings is demanding when it happens quickly and while multiple voices must be sustained. These comparisons

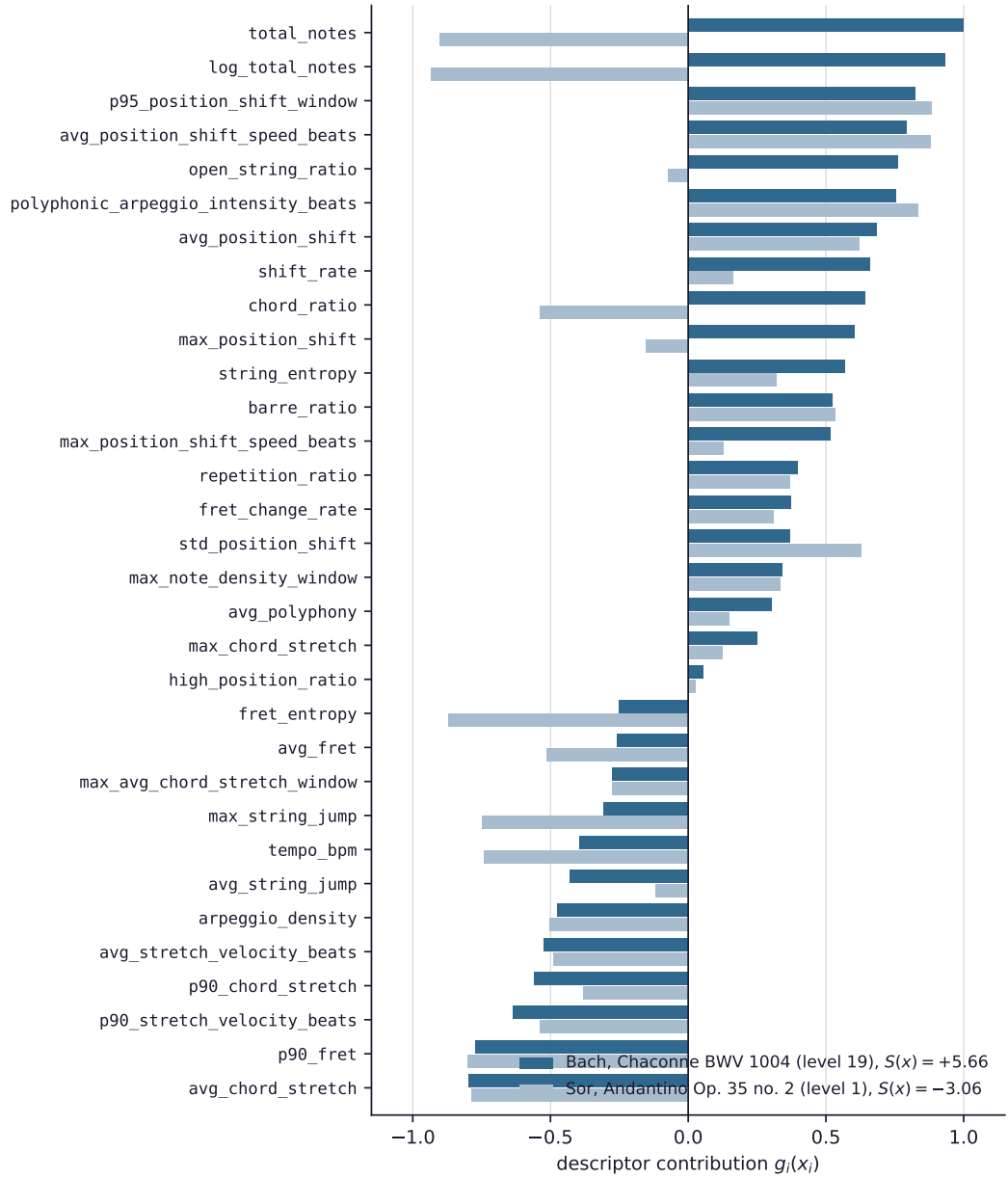


Figure 5.4: Per-descriptor contributions $g_i(x_i)$ for an easy and a hard test piece (V3 model, fold 0), sorted by the Chaconne's contributions. The summed scores $S(x) = -3.06$ and $+5.66$ place the pieces in classes 0 and 6, respectively; every bar is one line of the rubric behind those predictions.

Table 5.4: Context-free descriptors vs. their rhythm-aware counterparts: Spearman ρ against raw difficulty on the 639 pieces with rhythm data.

Context-free	ρ	Context-aware	ρ
avg_chord_stretch	+0.33	avg_stretch_velocity_beats	+0.40
		max_avg_chord_stretch_window	+0.39
avg_position_shift	+0.13	avg_position_shift_speed_beats	+0.30
		max_position_shift_speed_beats	+0.44
		p95_position_shift_window	+0.37
arpeggio_density	-0.14	polyphonic_arpeggio_intensity_beats	+0.22

confirm the design hypothesis and explain why the V3 generation helped the additive model in particular: the interaction between demand and available time is precisely what RubricNet cannot learn on its own.

5.3.4 Fuzzy Rules as Difficulty Descriptions

The quantitative comparison in Section 5.2 attributed the fuzzy classifiers’ deficit to their nominal treatment of the ordinal class structure, not to uninformative rules. Table 5.5 shows this directly: the five most relevant complete-search rules for the easiest class (0) and the hardest class (7), taken from fold 0.

The two rule sets are near mirror images: the easiest class is characterized by low fret positions, small stretches, and stable hand position, while the hardest class is characterized by their high-value counterparts plus sheer piece length. This mirrors the descriptor influence and monotonicity findings of Sections 5.3.1 and 5.3.2, obtained through an entirely different, non-neural method — further triangulating that these descriptors carry genuine difficulty signal rather than an artifact of RubricNet’s training.

The fuzzy pattern tree selected for class 7 in fold 0 ($d_{\max} = 3$) is

```
OR(
    AVG(max_position_shift IS high, total_notes IS very high),
```

Table 5.5: Top-5 complete-search rules by relevance (Equation 4.6) for the easiest and hardest difficulty classes, fold 0.

Class 0 (easiest)	Relevance
fret_entropy is very low	0.394
p90_fret is low	0.384
max_chord_stretch is low	0.371
avg_fret is very low	0.365
max_position_shift is very low	0.333
Class 7 (hardest)	Relevance
max_position_shift is high	0.394
fret_entropy is very high	0.371
p90_fret is very high	0.358
p95_position_shift_window is high	0.343
total_notes is very high	0.338

avg_string_jump IS moderate
)

which reads as: a piece is very likely class 7 if it combines frequent large position shifts with a high note count, *or* if the right hand jumps between strings at a moderate rate. Across all 40 induced trees (five classes $\times$ 5 folds excluding classes with too few positive examples in some tail folds), 28 (70%) contain at least one negated leaf, in clear contrast to Heerde et al., who found their evolutionary and pattern-tree rules never selected a negated operator [9]. On this dataset, negation is evidently useful — for instance, “total_notes is NOT low” rules out the easiest pieces without committing to a specific higher value, which the ablation in Section 5.4 confirms: disabling negated leaves lowers fuzzy pattern tree accuracy from 0.268 to 0.235 and Kendall’s τ from 0.484 to 0.385.

5.4 Auxiliary Experiments

Several further experiments were conducted after the main results were established. Most produced negative results; they are reported here because each one either justifies a design decision or marks off an unproductive direction.

5.4.1 Direct 20-Level Classification

The most natural objection to the 8-class formulation is that GuitarBurst’s native scale has 20 levels, raising the question of why those are not predicted directly. This was attempted, with considerable effort: training on the raw 1–20 targets initially failed to converge, and required monotonic initialization of the output layer (weights fixed at 1, biases stepping down by 0.5 per class) as well as disabling the inverse-frequency class weights — which, with only 3–8 pieces at levels 17–20, otherwise inject substantial gradient variance — before convergence was achieved at all. A dedicated 15-trial hyperparameter search improved validation accuracy from 0.168 to 0.303, but test accuracy still reached only 18.2%, with an MAE of 2.07 on the 20-level scale and Kendall’s τ of 0.43; even mapped down to the coarse 3-class view it achieved just 43.5%, far below the 67.6% of the 8-class model evaluated in the same way. The conclusion is that 716 pieces cannot support 20-way ordinal classification — the extreme classes are simply too sparse — and the 8-class binning of Section 4.1.4 is a necessity rather than a convenience.

5.4.2 Pruning Weak Descriptors

The V2 audit had flagged six of the surviving descriptors as the weakest (each $|\rho| \leq 0.16$): `tempo_bpm`, `arpeggio_density`, `fret_change_rate`, `avg_string_jump`, `chord_ratio`, and `avg_polyphony`. Since an additive model sums in every input, removing them appeared to be an obvious improvement. It was not: training RubricNet on the pruned 26-descriptor set *reduced* accuracy from 0.310 to 0.302, roughly doubled the fold-to-fold variance, and collapsed one fold to 0.196 accuracy. A plausible explanation is that the tuned regularization (dropout 0.20, weight decay 0.064) already suppresses uninformative subnetworks effectively, while the additional

inputs act as mild stabilizers; it may also matter that the hyperparameters were tuned for 32 inputs and reused unchanged for 26. In either case, the full descriptor set was retained.

5.4.3 Ordinal Label Smoothing

Sweeping the smoothing temperature over $T \in \{0.15, 0.3, 0.6\}$ against the frozen V3 hyperparameters showed a small, consistent improvement at $T = 0.15$ (accuracy 0.312, balanced accuracy 0.294, no fold collapse), while $T \geq 0.3$ was flat to worse — over-softening erodes the very boundary signal the ordinal loss requires. The improvement is within one standard deviation of the baseline, so it constitutes a direction rather than a result; the headline V3 numbers use the unsmoothed loss.

5.4.4 Multi-Task Coarse Auxiliary Head

A second output head predicting the 3-class grouping was trained jointly with the 8-class objective, sharing the aggregated score $S(x)$. All tested loss weightings underperformed the single-task baseline (accuracy 0.293–0.307 vs. 0.310), consistently rather than catastrophically. The diagnosis is structural: in a standard multi-task network, the auxiliary task regularizes a wide shared representation, but RubricNet’s entire shared trunk is a single scalar. The two heads can only compete over how that one number is shaped, and the coarse class boundaries do not coincide with the fine-grained thresholds.

5.4.5 Refined Left-Hand Descriptors (V4)

A fourth descriptor iteration refined the left-hand feature definitions and barre detection. It performed slightly *worse* than V3 across the board (accuracy 0.306, balanced accuracy 0.288, MAE 1.089), with identical coarse 3-class accuracy. Since it improved nothing while altering definitions that were already validated, V3 remains the final descriptor set, and V4 is noted here only for completeness.

5.4.6 Rhythm Ablation and Interaction Descriptors

After the main V3 results were established, the question arose whether further improvements were possible within the additive architecture. Two complementary hypotheses were tested: first, that the V3 rhythm-aware features might be adding noise rather than signal; second, that hand-crafted interaction descriptors (products or composites of existing features) could encode multi-factor difficulty that a single descriptor cannot capture.

Rhythm Ablation

To isolate rhythm’s contribution, a reduced descriptor set was formed by removing all eight rhythm-aware V3 features (Section 4.3.3), leaving the 24 non-rhythm descriptors of the V2 set. The Random Forest reached 0.325 accuracy on this reduced set (essentially unchanged from V3’s 0.331, within the margin of imputation variance), while RubricNet dropped to 0.301 from 0.310. This indicates that the rhythm features, while not decisive for the ensemble model, provide a modest but genuine signal contribution to RubricNet that the more flexible model can substitute for from other inputs.

Hand-Crafted Interaction Descriptors

Five composite descriptors were designed to capture multi-factor difficulty:

1. `barre_difficulty_tempo`: $\text{barre_ratio} \times (1.0 + \text{std_position_shift})$ — barres are harder when positions are unstable.
2. `stretch_under_time_pressure`: $\text{p90_chord_stretch} / (\text{min_inter-onset_interval} + 0.1)$ — large stretches are harder when little time is available.
3. `position_shift_entropy`: $\text{std_position_shift} \times \text{string_entropy}$ — frequent shifts combined with string variety compound the left-hand demand.
4. `open_string_efficiency`: $(1.0 - \text{open_string_ratio}) \times (\text{max_position_shift} + 1.0)$ — pieces that avoid open strings while shifting positions are harder.

5. `arpeggio_stretch_coupling`: `arpeggio_density` $\times$ `avg_chord_stretch` (if `arpeggio_density` > 0.1 , else 0) — arpeggios spanning wide stretches are demanding.

Each descriptor had a univariate Spearman $|\rho|$ in the range 0.21–0.46 against the raw 1–20 difficulty labels, indicating meaningful individual signal. However, when appended to the V3 base feature set, all three model families suffered consistent degradation, as Table 5.6 shows.

Table 5.6: Effect of interaction descriptors: RubricNet, ordinal regression, and Random Forest all declined when interaction features were appended to the V3 base descriptors.

Model	Baseline (V3 Base)	+ Interactions	Change
RubricNet	0.301 acc	0.285 acc	−0.016
Ordinal Regression	0.325 acc	0.282 acc	−0.043
Random Forest	0.325 acc	0.310 acc	−0.015

The root cause is architectural. Each interaction descriptor is a product (or multiplicative composite) of features already present in the set. In an additive model, appending such derived features creates collinearity: the product does not contribute independent information, but reweights its parent features through the nonlinearity. The regularization (dropout and ℓ_2 weight decay), tuned for the 20-descriptor base set, then struggles to suppress the noise in the new redundant dimensions, degrading generalization. Non-additive models can in principle exploit interactions through learned feature combinations; the strictly additive constraint prevents RubricNet from benefiting.

This experiment exposed a fundamental architectural limitation: **the additive model cannot be improved by appending collinear feature products**. Guitar difficulty does appear to depend on multi-factor interactions — stretch difficulty depends on available execution time, for instance — but capturing them within an additive setting requires either the hand-crafted context-aware descriptors already used in V3, which encode specific expected interactions without introducing collinear-

ity, or relaxing the additive constraint at the cost of interpretability. Given that the central claim of this thesis rests on interpretability, V3 remains the final model. The implications are discussed further in Section 6.4.

5.4.7 Fuzzy Rule Ablations

Three design choices behind the fuzzy classifiers of Section 4.5 were tested against alternatives, each on the full 5-fold protocol.

Normalization. Replacing the CDF-based fuzzification with plain min-max normalization slightly *improved* both methods (complete search: accuracy 0.271, τ 0.557 vs. 0.264/0.464; fuzzy pattern tree: accuracy 0.278, τ 0.533 vs. 0.268/0.484). This is a mild surprise given the heavy-tailed descriptors motivating the CDF choice; a plausible explanation is that min-max preserves absolute magnitude differences between pieces that the CDF’s rank-based mapping discards, and for a task with as few training pieces per fold as this one, that extra information outweighs the robustness benefit. The headline results nonetheless use the CDF variant, since it is the more faithful reading of evenly spaced linguistic terms on skewed features and the difference is within one standard deviation.

Balanced vs. plain RMSE. Switching fuzzy pattern tree induction from class-balanced to plain RMSE left accuracy essentially unchanged (0.278 vs. 0.268) while leaving τ comparable (0.537 vs. 0.484); no fold produced a degenerate constant-output tree under either setting. The imbalance in this task (up to $3.7\times$) is evidently mild enough that plain RMSE does not collapse to majority-class prediction here, unlike the more skewed one-vs-all splits anticipated in Section 4.5. The balanced variant is kept as the default since it carries no measurable cost and guards against the failure mode on splits where it would matter.

Negated leaves. Disabling negation in fuzzy pattern tree induction is the one ablation with a clear, substantial effect: accuracy drops from 0.268 to 0.235, MAE rises from 1.469 to 1.754, and τ falls from 0.484 to 0.385. Unlike Heerde et al., who found negated operators were never selected in their genre-classification setting [9], 70% of the trees induced here use at least one negated leaf (Section 5.3.4). Negation is therefore not merely a faithfulness extension of the source paper but a genuinely

useful addition on this dataset, and is kept enabled by default.

Chapter 6

Discussion

The discussion below examines how the proposed approach answers the research questions posed in Section 1.3, drawing on the quantitative and qualitative results of Chapter 5, and addresses the limitations that qualify each answer.

6.1 Descriptor Design

The first research question asked which guitar-specific symbolic descriptors best capture the technical difficulty of classical guitar pieces. The audits of Section 4.3 and the qualitative analysis of Section 5.3 give a consistent answer: piece length, fretboard coverage and position statistics (`fret_entropy`, `high_position_ratio`, `avg_fret`), shift frequency, and — once rhythm is available — demand-per-beat measures carry the strongest signal, with open-string content as the strongest negative indicator. Equally informative is what did *not* work: several intuitively plausible descriptors, including raw shift distance, string-jump statistics, and polyphony averages, proved nearly uncorrelated with graded difficulty until they were placed in temporal context. The comparison of Section 5.3.3 shows why: a physical demand only becomes difficult in proportion to the time available to execute it, so context-free averages conflate leisurely and virtuosic instances of the same motion.

These findings both extend and depart from the prior work on guitar playability. The descriptors of Vélez Vázquez et al. [25] — barre usage, chord-shape displacement — have direct counterparts in our set, but the strongest descriptors for classical

repertoire turned out to be ones with no analogue in strummed chord charts: fret-board coverage entropy and rhythm-normalized movement statistics, reflecting the horizontal, position-based way the classical left hand traverses the neck.

Three limitations qualify the descriptor layer. First, tablature commits each note to a string and fret, which the descriptors take at face value; where a transcription chose an awkward fingering, the descriptors measure the transcription, not the piece’s best playable realization. A fingering-optimization step (Section 7.1) would decouple the two. Second, the rhythm-alignment pipeline collapses multi-voice guitar textures into a single stream of note attacks: pitch content and onset positions — what the descriptors actually consume — are correct, but the notated voice-leading structure is lost, so descriptors that would distinguish a sustained bass line under a melody from block writing are out of reach for the PDF-derived majority of the dataset. Third, rhythm recovery requires a matched MIDI performance per PDF score; for 76 of the PDF scores the alignment coverage was too low or no matching MIDI existed, and those pieces fall back to median-imputed rhythm descriptors. As built, the system cannot grade an arbitrary new PDF score without either native rhythm information or a human-in-the-loop MIDI match.

6.2 Predictive Performance

The second research question asked whether an interpretable, rubric-based model can predict piece-level difficulty with useful accuracy. The results of Section 5.2 support an affirmative answer with a precise qualification. On raw exact accuracy, the Random Forest retains a two-point advantage (0.331 vs. 0.310); on every metric that respects the ordinal structure of the task — balanced accuracy, MAE, MSE, and Kendall’s τ — the additive model outperforms it. When RubricNet errs, it errs by less, and it orders pieces by difficulty more faithfully. Since the practical use case is ordinal through and through (a one-bin miss sends a student a slightly easy piece; a four-bin miss sends a beginner a concert work), the error and ranking metrics are the appropriate standard, and by that standard the interpretability of the additive model costs essentially nothing.

The fuzzy rule-based comparison sharpens this answer from the other side. The complete search and fuzzy pattern tree classifiers are as transparent as RubricNet — arguably more immediately readable, since their output is natural-language rules — and they operate on the identical V3 descriptors, yet they trail it on every metric, most severely on ranking quality (τ 0.464 and 0.484 vs. 0.634). Since Section 5.3.4 shows their induced rules capture genuine difficulty structure, the deficit lies in their nominal one-vs-all decision rule, which carries no notion that class 6 is closer to class 7 than class 1 is. RubricNet’s standing is therefore not evidence that interpretability comes cheap in general; it is evidence for the specific combination of an interpretable architecture with an ordinal design. Interpretability alone, without ordinal awareness, measurably leaves performance on the table.

The absolute numbers warrant a candid reading. At 31.0% exact accuracy over 8 classes (67.6% over three levels, with 73.3% of predictions within one bin), the model is not a replacement for human grading; it functions as a transparent second opinion and pre-filter over large score collections. Two considerations frame these figures. First, the failed direct 20-level experiment (Section 5.4.1) shows that the ceiling is partly a data problem: 716 pieces cannot populate the extremes of a 20-level scale, and even the 8-class formulation leaves classes with as few as 37 examples. Second, all grades originate from a single database, effectively a single grading tradition. The manual reviews found the grades plausible, but the model necessarily inherits any systematic bias in GuitarBurst’s grading, and the reported accuracies are relative to that single standard rather than to teacher consensus at large. Part of the residual error may therefore reflect noise in the labels themselves rather than deficiencies of the model.

A further observation from Table 5.1 deserves emphasis because it generalizes beyond this thesis: every substantial performance gain came from better data and better descriptors — repairing a corrupted extraction path, recovering rhythm, encoding demand-versus-time interactions by hand — while the model itself never changed. For interpretable-by-construction architectures, whose representational capacity is deliberately fixed, the descriptors effectively *are* the model.

6.3 The Multi-Dimensional Nature of Guitar Difficulty

The third research question asked whether combining left-hand, right-hand, and global descriptor dimensions improves prediction over any single dimension alone. The ablation of Section 5.2.2 answers affirmatively: the full set outperforms every individual dimension on every metric. The left hand is the strongest single dimension, consistent with the pedagogical view that stretches, shifts, and barre work dominate classical guitar difficulty, yet even it falls measurably short of the combination; the right-hand dimension, weak in isolation, still contributes signal the other dimensions do not carry. This mirrors, for the guitar, the finding of Ramoneda et al. [16] that piano difficulty is best modeled as a combination of separate dimensions rather than a single pooled representation, and suggests that multi-dimensionality is a property of instrumental difficulty as such rather than of any particular instrument.

6.4 Interpretability

The fourth research question asked to what extent the model’s descriptor contributions align with pedagogical intuition. Four strands of evidence from Section 5.3 bear on this. The influence analysis shows that the descriptors doing the most work — piece length, shift frequency, fretboard coverage, open-string content with a negative sign — are those a teacher would name unprompted. The monotonicity analysis shows that the influential descriptors’ contributions rise (or, for `open.string_ratio`, fall) steadily across the eight difficulty classes, so a between-class difference decomposes into descriptor movements a teacher can verify against the score. The triangulation against Random Forest importances and model-free rank correlations shows that this learned emphasis is not an artifact of RubricNet’s own training. And the fuzzy rules of Section 5.3.4 corroborate the same story from an entirely different model family: a deterministic, non-neural rule search, given the same descriptors, independently selects low fretboard position, small stretches, and positional stability as the signature of the easiest pieces, and their opposites plus high note count for the hardest. Taken

together, these results indicate that the interpretability is substantive: the trained model can be read as a difficulty rubric grounded in guitar technique.

The interpretability has a precise cost, and the interaction-descriptor experiment of Section 5.4.6 locates it. The additive constraint prevents the model from learning interactions between descriptors, and appending generic interaction products does not recover them — such products merely introduce collinearity and degrade generalization, a failure observed across all model families tested. The only route to interaction effects that worked within the additive setting is the one taken by the V3 context-aware descriptors: encoding a specific, pedagogically motivated interaction (demand per unit time) directly into a single input. Relaxing the additive constraint into a standard neural network would likely raise raw accuracy, but at the cost of the decomposable predictions that motivate the entire approach; given the purpose of the model, the constraint is retained. It should also be noted that whether practicing teachers find the per-piece rubric decompositions convincing is an empirical question that this thesis does not answer; the evidence presented here establishes internal consistency and agreement with independent statistical measures, and an external validation with teachers remains future work.

Chapter 7

Conclusion

This thesis set out to build a difficulty estimator for classical guitar that a teacher could not only use but also interrogate. To this end, it adapted the additive Rubric-Net architecture from piano to classical guitar, guided by four research questions concerning descriptor design, predictive performance under an interpretability constraint, the multi-dimensional structure of difficulty, and the pedagogical validity of the model’s explanations. The work makes four contributions:

1. **A labeled symbolic dataset for classical guitar difficulty.** The dataset comprises 716 unique pieces pairing symbolic scores from three heterogeneous sources with professional 1–20 difficulty grades, assembled through fuzzy matching, manual auditing, and deduplication. Since the largest source (ClassClef PDF scores) carries no rhythm, a Needleman–Wunsch alignment pipeline transplants timing from matched MIDI performances onto the PDF-derived pitch sequences, leaving 89.2% of the dataset with complete note durations.
2. **A guitar-specific descriptor set, developed and audited across three generations.** Starting from 12 straightforward descriptors, correlation audits exposed uninformative features and extraction defects; the final set of 32 descriptors spans left-hand technique, right-hand technique, and global score complexity, and includes rhythm-aware descriptors — windowed bottleneck measures and demand-per-beat proxies — that encode the interaction between physical difficulty and available execution time.

3. **The first adaptation of the additive RubricNet architecture beyond piano.** The final model reaches 31.0% exact and 29.1% balanced accuracy on the 8-class task (67.6% on a coarse 3-level scale) and outperforms a Random Forest on balanced accuracy, MAE, MSE, and Kendall’s τ , while every prediction remains decomposable into per-descriptor contributions. Two fuzzy rule-based classifiers — a complete search over primitive rules and fuzzy pattern trees, adapted here from music genre classification to difficulty grading — serve as a second interpretable reference point; the model outperforms both on every metric, isolating the value of its ordinal design.
4. **Evidence that the interpretability is substantive.** Descriptor contributions increase monotonically with difficulty, the model’s influence rankings agree with Random Forest importances and model-free rank correlations, and the learned signs match pedagogical expectations — open-string content lowers predicted difficulty, shift frequency raises it.

Returning to the research questions: the descriptors that best capture graded difficulty are piece length, fretboard coverage and position statistics, shift frequency, and — once rhythm is available — demand-per-beat measures, with open-string content as the strongest negative indicator. A rubric-based model over these descriptors predicts difficulty with useful accuracy, matching a black-box baseline on ranking quality and surpassing it on error distance; equally interpretable fuzzy rule-based classifiers without ordinal awareness trail it by a wide margin on the same descriptors, showing that the combination of interpretability and ordinal design, not interpretability alone, is what keeps the cost of transparency near zero. Combining the three technical dimensions outperforms every single dimension alone, with the left hand strongest in isolation and each dimension contributing to the whole. Finally, the model’s explanations align with pedagogical intuition, as demonstrated by the monotonicity and triangulation analyses.

A finding that runs through the entire project deserves restating: every substantial performance gain came from better data and better descriptors — repairing a corrupted extraction path, recovering rhythm, encoding demand-versus-time interactions

by hand — while the model itself never changed. For interpretable-by-construction architectures, the descriptors are the model.

7.1 Future Work

There are several avenues for future work.

One direction is to improve the treatment of pieces without rhythm information. The 10.8% of pieces without rhythm currently receive training-fold median values for the eight rhythm-aware descriptors — a safe but coarse fallback. A model trained on the rhythm-complete majority to predict rhythm descriptors from pitch-and-fret content could replace the medians with piece-specific estimates. A related and cheaper experiment worth conducting first is a re-evaluation on the rhythm-complete subset alone, to quantify exactly how much of V3’s gain the imputed pieces dilute.

A second direction concerns the alignment pipeline’s single-voice output, which discards notated voice structure. Splitting scores into melody and bass streams before alignment — for example with the voice-separation utilities of the music21 toolkit [6] — would preserve sustained bass durations and enable contrapuntal-complexity descriptors, an aspect of classical guitar difficulty the current set does not address.

Third, the descriptors currently trust the string-and-fret choices baked into each transcription. A guitar fingering solver that computes an ergonomic left-hand path from pitch content alone would let the descriptors measure a piece’s *best-case* physical difficulty rather than one transcriber’s choices, and would additionally enable the grading of scores distributed as plain staff notation without tablature.

Fourth, the interaction-descriptor experiments showed that although guitar difficulty depends on multi-factor interactions, generic feature products only introduce collinearity into an additive model. A promising direction is decorrelated pairwise interactions, in the spirit of GA²M [13]: rather than appending descriptor products directly, each candidate interaction could be residualized against its parent features, and the residuals — statistically independent of the existing inputs — fed to an otherwise-additive model. This would expand the interpretable model’s capacity for

interaction effects while preserving the rubric-like decomposition of its predictions.

Fifth, the fuzzy rule-based baselines invite an ordinal-aware variant. Their deficit was traced to the nominal one-vs-all decision rule rather than to the induced rules themselves, so decoding an expected grade from the per-class rule activations — instead of taking their argmax — could close much of the gap while retaining natural-language rules as the explanation format. The evolutionary rule-base induction of Heerde et al. [9], excluded here for its nondeterminism, likewise remains untested on difficulty grading.

Finally, all results in this thesis are relative to a single grading database. The natural next step is external validation: comparing the model’s rubrics against independent grading systems, and placing the per-piece descriptor score sheets in front of practicing teachers to test whether the explanations change or support their judgments. The additive architecture makes such a study directly possible — each prediction already constitutes an itemized rubric — but whether teachers find the itemization convincing is an empirical question that remains open.

Bibliography

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019.
- [2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [3] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, Belmont, CA, 1984.
- [4] Kay H. Brodersen, Cheng Soon Ong, Klaas E. Stephan, and Joachim M. Buhmann. The balanced accuracy and its posterior distribution. In *Proceedings of the 20th International Conference on Pattern Recognition (ICPR)*, pages 3121–3124. IEEE, 2010.
- [5] Jianlin Cheng, Zheng Wang, and Gianluca Pollastri. A neural network approach to ordinal regression. In *Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN)*, pages 1279–1284, 2008.
- [6] Michael Scott Cuthbert and Christopher Ariza. music21: A toolkit for computer-aided musicology and symbolic music data. In *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*, pages 637–642, 2010.
- [7] Michael Good. MusicXML: An internet-friendly format for sheet music. In *Proceedings of the XML Conference and Expo*, pages 3–4, 2001.

- [8] GuitarBurst. Guitarburst: Graded classical guitar repertoire database. <https://guitarburst.com>, 2021. Accessed: 2026-05-04.
- [9] Frederik Heerde, Igor Vatulkin, and Günter Rudolph. Comparing fuzzy rule based approaches for music genre classification. In *Proceedings of the 9th International Conference on Artificial Intelligence in Music, Sound, Art and Design (EvoMUSART)*, volume 12103 of *LNCS*, pages 35–48. Springer, 2020.
- [10] Zhiheng Huang, Tam D. Gedeon, and Masoud Nikraves. Pattern trees induction: A new machine learning method. *IEEE Transactions on Fuzzy Systems*, 16(4):958–970, 2008.
- [11] Maurice G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1/2):81–93, 1938.
- [12] Maurice G. Kendall. The treatment of ties in ranking problems. *Biometrika*, 33(3):239–251, 1945.
- [13] Yin Lou, Rich Caruana, Johannes Gehrke, and Giles Hooker. Accurate intelligible models with pairwise interactions. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 623–631, 2013.
- [14] Saul B. Needleman and Christian D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3):443–453, 1970.
- [15] Pedro Ramoneda, Vsevolod Eremenko, Alexandre D’Hooge, Emilia Parada-Cabaleiro, and Xavier Serra. Towards explainable and interpretable musical difficulty estimation: A parameter-efficient approach. In *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*, pages 520–528, 2024.
- [16] Pedro Ramoneda, Dasaem Jeong, Vsevolod Eremenko, Nazif Can Tamer, Marius Miron, and Xavier Serra. Combining piano performance dimensions for score difficulty classification. *Expert Systems with Applications*, 238:121776, 2024.

- [17] Xavier Riley, Zixun Guo, Drew Edwards, and Simon Dixon. Gaps: A large and diverse classical guitar dataset and benchmark transcription model. In *Proceedings of the 25th International Society for Music Information Retrieval Conference (ISMIR)*, 2024.
- [18] Pedro Sarmiento, Adarsh Kumar, CJ Carr, Zack Zukowski, Mathieu Barthet, and Yi-Hsuan Yang. Dadagp: A dataset of tokenized guitarpro songs for sequence models. In *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*, pages 610–617, 2021.
- [19] Robin Senge and Eyke Hüllermeier. Top-down induction of fuzzy pattern trees. *IEEE Transactions on Fuzzy Systems*, 19(2):241–252, 2011.
- [20] Robin Senge and Eyke Hüllermeier. Fast fuzzy pattern tree learning for classification. *IEEE Transactions on Fuzzy Systems*, 23(6):2024–2033, 2015.
- [21] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [22] Charles Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904.
- [23] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.
- [24] Igor Vatolkin and Günter Rudolph. Interpretable music categorisation based on fuzzy rules and high-level audio features. In *Data Science, Learning by Latent Structures, and Knowledge Discovery*, Studies in Classification, Data Analysis, and Knowledge Organization, pages 423–432. Springer, 2015.
- [25] Marcel A. Vélez Vásquez, Mariëlle Baelemans, Jonathan Driedger, Willem Zuidema, and John Ashley Burgoyne. Quantifying the ease of playing song chords on the guitar. In *Proceedings of the 24th International Society for Music Information Retrieval Conference (ISMIR)*, pages 725–732, 2023.

- [26] Eli J. Yaroch. Measure by measure: Developing an artificial intelligence algorithm for music difficulty assessment. Master's research project, University of Miami, 2025.
- [27] Lotfi A. Zadeh. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965.